Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Информационная безопасность

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:

Разработка методики верификации соответствия TLA+ спецификации и исходного кода

Студент ИУ8-124 (Группа) _____ (Подпись, дата) Н. В. Железцов (И.О.Фамилия)

Руководитель ВКР _____ (Подпись, дата) Н. В. Медведев (И.О.Фамилия)

Консультант по исследовательской части _____ (Подпись, дата) Н. В. Медведев (И.О.Фамилия)

Консультант по конструкторской части _____ (Подпись, дата) Н. В. Медведев (И.О.Фамилия)

Консультант по технологической части _____ (Подпись, дата) Н. В. Медведев (И.О.Фамилия)

Консультант по организационно-экономической части _____ (Подпись, дата) Н.Е. Беняев (И.О.Фамилия)

Консультант по организационно-правовому обеспечению ИБ _____ (Подпись, дата) И. Ю. Шахалов (И.О.Фамилия)

Нормоконтролер _____ (Подпись, дата) Т.Е. Завадская (И.О.Фамилия)

2026 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой ИУ8
(Индекс)
_____ М.А.Басараб
(И.О.Фамилия)
« _____ » _____ 20 ____ г.

З А Д А Н И Е
на выполнение выпускной квалификационной работы

Студент группы ИУ8-114 Железцов Никита Владимирович
(фамилия, имя, отчество)

Тема квалификационной работы

Разработка методики верификации соответствия TLA+ спецификации и исходного кода

При выполнении ВКР:

Используются / Не используются	Да / Нет
1) Литературные источники и документы, имеющие гриф секретности	Нет
2) Литературные источники и документы, имеющие пометку «Для служебного пользования», иных пометок, запрещающих открытое опубликование	Нет
3) Служебные материалы других организаций	Нет
4) Результаты НИР (ОКР), выполняемой в МГТУ им.Н.Э.Баумана	Нет
5) Материалы по незавершенным исследованиям или материалы по завершенным исследованиям, но ещё не опубликованные в открытой печати	Нет

Тема квалификационной работы утверждена распоряжением по факультету ИУ № _____ от « ___ » _____ 20__ г.

Часть 1. (Исследовательская)
- описание модели распределенной системы;
- обзор и анализ существующих алгоритмов консенсуса, обеспечивающих согласованность данных и состояний между узлами распределенной системы;
- обзор и анализ существующих подходов к верификации соответствия TLA+ спецификации и исходного кода;
- формулировка математической постановки задачи;
- обоснование решения математической постановки задачи;

Часть 2. (Конструкторская)
- обоснование выбора алгоритма консенсуса;
- обоснование выбора языка программирования и инструментов реализации;
- проектирование архитектуры системы распределенных вычислений с использованием выбранного алгоритма консенсуса;
- разработка схемы взаимодействия между частями системы верификации соответствия TLA+ спецификации и исходного кода;

Часть 3. (Технологическая)
- разработка и тестирование TLA+ спецификации выбранного алгоритма консенсуса;
- разработка и тестирование системы распределенных вычислений с использованием выбранного алгоритма консенсуса, реализуемого на основе разработанной TLA+ спецификации;
- разработка и интеграция системы верификации соответствия TLA+ спецификации в исходный код системы распределенных вычислений;
- анализ результатов интеграции;

Часть 4. (Организационно — экономическая)
- описание затрат времени и необходимых материальных ресурсов для реализации проекта;
- обоснование экономической целесообразности выпускной квалификационной работы.

Часть 5. (Организационно – правовое обеспечение ИБ)
– анализ действующих нормативных правовых актов Российской Федерации, регулирующих правовое обоснование выполнения выпускной квалификационной работы.

Оформление квалификационной работы:
Расчетно-пояснительная записка на _____ листах формата А4.
Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т. п.):
презентация не менее 14 слайдов, схемы, диаграммы и рисунки.
Дата выдачи задания « 01 » декабря 2025 г.
В соответствии с учебным планом выпускную квалификационную работу выполнить в полном объеме в срок до « 31 » мая 2026 г.

Консультант по исследовательской части	_____ (Подпись, дата)	Н. В. Медведев (И.О.Фамилия)
Консультант по конструкторской части	_____ (Подпись, дата)	Н. В. Медведев (И.О.Фамилия)
Консультант по технологической части	_____ (Подпись, дата)	Н. В. Медведев (И.О.Фамилия)
Консультант по организационно-экономической части	_____ (Подпись, дата)	Н. Е. Беняев (И.О.Фамилия)
Консультант по организационно-правовому обеспечению ИБ	_____ (Подпись, дата)	И. Ю. Шахалов (И.О.Фамилия)
Руководитель квалификационной работы	_____ (Подпись, дата)	Н. В. Медведев (И.О.Фамилия)
Студент	_____ (Подпись, дата)	Н.В. Железцов (И.О.Фамилия)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ

КАФЕДРА ИУ8

ГРУППА ИУ8-114

УТВЕРЖДАЮ

Заведующий кафедрой ИУ8
(Индекс)

_____ М.А.Басараб
(И.О.Фамилия)

« _____ » _____ 20 ____ г.

КАЛЕНДАРНЫЙ ПЛАН
выполнения выпускной квалификационной работы

студента группы ИУ8-114 Железцова Никиты Владимировича
(фамилия, имя, отчество)

Тема квалификационной работы

Разработка методики верификации соответствия TLA+ спецификации и исходного кода

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Должность	ФИО, подпись
1.	Формулирование проблемы, цели и задач работы	*20.12.25*		Руководитель ВКР	
2.	Описание модели распределенной системы; обзор и анализ существующих алгоритмов консенсуса, обеспечивающих согласованность данных и состояний между узлами распределенной системы	*07.02.26*		Консультант по исследовательской части	
3.	Обзор и анализ существующих подходов к верификации соответствия TLA+ спецификации и исходного кода	*22.02.26*		Консультант по исследовательской части	
4.	Формулировка и обоснование решения математической постановки задачи	*05.03.26*		Консультант по исследовательской части	
5.	Утверждение окончательных формулировок решаемой проблемы, цели работы и перечня задач	*06.03.26*		Заведующий кафедрой	

6.	Обоснование выбора алгоритма консенсуса, языка программирования и инструментов реализации	*15.03.26*		Консультант по конструкторской части	
7.	Проектирование архитектуры системы распределенных вычислений с использованием выбранного алгоритма консенсуса	*22.03.26*		Консультант по конструкторской части	
8.	Разработка схемы взаимодействия между частями системы верификации соответствия TLA+ спецификации и исходного кода	*29.03.26*		Консультант по конструкторской части	
9.	Разработка и тестирование TLA+ спецификации выбранного алгоритма консенсуса	*05.04.26*		Консультант по технологической части	
10.	Разработка и тестирование системы распределенных вычислений с использованием выбранного алгоритма консенсуса	*12.04.26*		Консультант по технологической части	
11.	Разработка и интеграция системы верификации соответствия TLA+ спецификации в исходный код системы распределенных вычислений, анализ результатов	*19.04.26*		Консультант по технологической части	
12.	Организационно-экономическая часть	*20.04.26*		Консультант по организационно-экономической части	
13.	Организационно-правовая часть	*21.04.26*		Консультант по организационно-правовой части	
14.	1-я редакция работы	*22.04.26*		Руководитель ВКР	
15.	Подготовка доклада и презентации	*24.04.26*		Руководитель ВКР	
16.	Предзащита ВКР				
17.	Корректировка ВКР по результатам предзащиты	*05.05.26*			
18.	Заключение руководителя	*11.05.26*		Руководитель ВКР	
19.	Нормконтроль	*13.05.26*		Нормоконтролер	
20.	Внешняя рецензия	*10.06.26*			
21.	Защита работы на ГЭК				

Студент _____
(подпись, дата)

Руководитель работы _____
(подпись, дата)

РЕФЕРАТ

Отчёт содержит 93 стр., 9 рис., 16 табл., 32 источн.

Ключевые слова: TLA+, МОДЕЛЬНОЕ ТЕСТИРОВАНИЕ, ФОРМАЛЬНАЯ ВЕРИФИКАЦИЯ, ТРАССИРОВКА, RAFT, РАСПРЕДЕЛЁННЫЕ СИСТЕМЫ.

Объектом исследования является соответствие TLA+ спецификации и исходного кода распределённой системы.

Цель работы — разработка методики верификации соответствия TLA+ спецификации и исходного кода на основе анализа трасс исполнения и её апробация на распределенной системе.

В работе предложена методика сопоставления трасс исполнения программы с формальной спецификацией, включающая инструментирование реализации, построение и объединение трасс, а также их проверку средствами TLA+ и TLC.

В результате разработаны TLA+ спецификация и программная реализация распределённой системы на основе алгоритма консенсуса Raft, а также интегрирована система проверки соответствия, позволяющая выявлять расхождения между формальной моделью и реальным исполнением программы.

СОДЕРЖАНИЕ

РЕФЕРАТ . . . 6
ВВЕДЕНИЕ . . . 9
ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ . . . 13
1 Модель распределённой системы . . . 14
1.1 Модель каналов связи . . . 15
1.2 Консенсус в распределённой системе . . . 16
1.3 Временная модель и ограничения достижимости консенсуса . . . 17
1.4 Модели отказов . . . 17
1.5 Принятые в работе допущения . . . 18
2 Обзор и анализ существующих алгоритмов консенсуса . . . 20
2.1 Алгоритм Paxos . . . 20
2.2 Алгоритм Multi-Paxos . . . 22
2.3 Алгоритм Raft . . . 23
2.4 Сравнительный анализ . . . 25
3 Теоретические основы системы верификации . . . 28
3.1 Формальная спецификация на языке TLA+ . . . 28
3.2 Проверка трасс исполнения по формальной модели . . . 29
3.3 Существующие подходы . . . 30
4 Математическая постановка задачи . . . 33
4.1 Формализация модели и ее поведения . . . 33
4.2 Формализация трассы программы . . . 34
4.3 Согласованность трассы с моделью . . . 35
4.4 Показатель качества согласованности . . . 37
4.5 Итоговая математическая постановка задачи . . . 38
КОНСТРУКТОРСКАЯ ЧАСТЬ . . . 39
ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ . . . 40
ОРГАНИЗАЦИОННО-ЭКОНОМИЧЕСКАЯ ЧАСТЬ . . . 41
5 Описание процесса изготовления продукта . . . 42
6 Диаграмма Ганта и сетевой график изготовления продукта . . . 48
7 Анализ затрат на реализацию проекта . . . 52
7.1 Анализ затрат на изготовление продукта . . . 52
7.2 Анализ затрат на внедрение продукта . . . 57
7.3 Общие затраты . . . 59

8 Исследование рынка 62
9 Планирование цены и прогнозирование прибыли 64
10 Цифровая оценка инвестиций в инновационное производство . . 69
11 Заключение 75
ОРГАНИЗАЦИОННО-ПРАВОВАЯ ЧАСТЬ 77
12 Конституция Российской Федерации 79
13 Федеральный закон от 27.07.2006 №149-ФЗ «Об информации, информационных технологиях и о защите информации» 82
14 Указ Президента Российской Федерации от 05.12.2016 №646 «Об утверждении Доктрины информационной безопасности Российской Федерации» 84
15 ГОСТ Р 56939–2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования» . . . 86
16 Приказ ФСТЭК России от 01.12.2023 №240 «Об утверждении Порядка проведения сертификации процессов безопасной разработки программного обеспечения средств защиты информации» 88
17 Заключение 89
ЗАКЛЮЧЕНИЕ 90
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 91

ВВЕДЕНИЕ

Мир вычислительных технологий за последние десятилетия претерпел существенные изменения. На смену централизованным и монолитным приложениям пришли распределённые программные системы, в которых обработка данных, хранение состояния и выполнение вычислений осуществляются множеством взаимодействующих узлов. Такой подход позволяет масштабировать вычислительные ресурсы, повышать доступность сервисов и обеспечивать устойчивость к частичным отказам оборудования и сетевой инфраструктуры.

Распределённые системы лежат в основе современных вычислительных платформ, облачных сервисов, систем хранения данных и транзакционных приложений. Однако их проектирование и реализация связаны с рядом фундаментальных трудностей. Поведение такой системы формируется в условиях асинхронности, конкурентного выполнения, недетерминированной доставки сообщений и возможных отказов отдельных процессов и каналов связи. В этих условиях даже небольшая ошибка в алгоритме координации может приводить к расхождению состояний реплик, потере данных, некорректному выбору лидера или нарушению свойств безопасности.

Ключевую роль в обеспечении согласованности данных и состояний между узлами распределённой системы играют алгоритмы консенсуса. Именно они позволяют множеству процессов прийти к согласованному решению относительно порядка команд, состояния журнала или результата взаимодействия, несмотря на задержки сообщений и сбои части узлов. Среди невизантийских алгоритмов консенсуса наибольшее практическое значение получили Paxos и Raft. При этом алгоритм Raft представляет особый интерес благодаря более ясной структуре, декомпозиции на подзадачи выбора лидера, репликации журнала и обеспечения безопасности, а также удобству практической реализации.

Для повышения надёжности распределённых алгоритмов всё шире применяются формальные методы. Одним из наиболее известных и практически значимых средств формального описания является язык спецификаций TLA+, позволяющий задавать систему как множество состояний и допустимых переходов между ними, а также формулировать инварианты

и свойства корректности. Использование средства проверки моделей TLC делает возможным автоматическое исследование пространства состояний спецификации и выявление нарушений сформулированных свойств ещё на этапе проектирования.

Однако наличие корректной формальной спецификации само по себе не гарантирует корректности программной реализации. Между абстрактной моделью и исходным кодом существует естественный разрыв. Спецификация описывает поведение на уровне состояний и переходов, тогда как реальная система включает сетевое взаимодействие, внутренние буферы, многопоточность, обработку ошибок, особенности используемых библиотек и инфраструктурные детали. В процессе развития проекта исходный код может изменяться быстрее, чем формальная модель, что создаёт риск расхождения между фактическим поведением программы и поведением, допускаемым спецификацией.

В связи с этим особую актуальность приобретает задача верификации соответствия TLA+ спецификации и исходного кода. Практически значимым подходом к решению данной задачи является валидация трасс исполнения программы относительно формальной модели. В рамках такого подхода реализация инструментируется таким образом, чтобы в ходе исполнения фиксировались наблюдаемые изменения состояния, имеющие смысл в терминах спецификации. Полученные локальные события объединяются в общую трассу, после чего проверяется, может ли данная трасса быть интерпретирована как допустимая последовательность переходов формальной модели. Тем самым проверяется уже не только корректность самой спецификации, но и соответствие ей поведения реальной исполняемой системы.

Обоснование актуальности темы исследования

Актуальность темы обусловлена одновременно теоретическими и практическими причинами. С теоретической точки зрения распределённые системы остаются одной из наиболее сложных областей программной инженерии, поскольку требуют учёта моделей отказов, ограничений синхронности, свойств каналов связи и механизмов достижения консенсуса.

С практической точки зрения ошибки в распределённых алгоритмах могут приводить к серьёзным последствиям: нарушению согласованности данных, остановке сервисов, потере результатов вычислений и необходимости дорогостоящей доработки системы на поздних этапах жизненного цикла.

Формальная верификация позволяет существенно повысить уверенность в корректности проектируемых решений. Вместе с тем наибольшую ценность она приобретает тогда, когда формальная модель не существует отдельно от реальной реализации, а используется как критерий проверки фактического поведения программы. Поэтому разработка методики, обеспечивающей систематическое сопоставление TLA+ спецификации и исходного кода, является актуальной задачей, имеющей значение как для исследовательской деятельности, так и для практики создания надёжного программного обеспечения.

Дополнительную значимость работе придаёт то, что предложенный подход рассматривается на примере распределённой системы вычислений, использующей алгоритм консенсуса Raft. Это позволяет не ограничиваться абстрактной постановкой задачи, а продемонстрировать применимость методики в условиях реальной распределённой реализации, включающей взаимодействие узлов, обмен сообщениями, выбор лидера, репликацию состояния и обработку отказов.

Цель и задачи работы

Целью выпускной квалификационной работы является разработка методики верификации соответствия TLA+ спецификации и исходного кода, а также её апробация на распределённой системе вычислений, использующей алгоритм консенсуса Raft.

Для достижения поставленной цели в работе решаются следующие задачи:

- проанализировать алгоритмы консенсуса и подходы к верификации соответствия TLA+ спецификации и исходного кода;
- сформулировать и обосновать методику верификации соответствия спецификации и реализации;
- спроектировать и реализовать распределённую систему вычислений;
- разработать и интегрировать систему верификации;
- оценить экономические и организационно-правовые аспекты разработки.

Объект и предмет исследования

Объектом исследования являются распределённые системы вычислений, использующие алгоритмы консенсуса для обеспечения согласованности состояния между узлами.

Предметом исследования является методика верификации соответствия TLA+ спецификации и исходного кода, основанная на инструментировании реализации, построении трасс исполнения и их последующей проверке средствами модельной проверки.

Методы исследования

В работе используются методы теории распределённых систем, формальной спецификации и модельной проверки, элементы теории переходных систем, подходы к инструментированию программных реализаций, а также методы экспериментальной апробации программных средств на тестовых сценариях работы распределённой системы.

Практическая значимость работы

Практическая значимость работы заключается в том, что предложенная методика позволяет обнаруживать расхождения между формальной моделью и реальным поведением программы на основе анализа трасс исполнения. Результаты работы могут быть использованы при разработке и сопровождении распределённых систем, для которых критически важны согласованность состояния, корректность координации и своевременное выявление логических ошибок реализации.

Кроме того, разработанные спецификация, программная реализация и средства интеграции проверки соответствия могут служить основой для дальнейшего расширения рассматриваемого подхода, включая применение к другим алгоритмам консенсуса и другим классам распределённых систем.

ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ

В исследовательской части рассматриваются теоретические основы разрабатываемой системы распределённых вычислений и методики верификации соответствия TLA+ спецификации и исходного кода. Сначала описывается модель разрабатываемой системы распределённых вычислений, включающая узлы, каналы связи, допустимые отказы и условия поддержания согласованного состояния. Затем проводится обзор и сравнительный анализ существующих алгоритмов консенсуса, обеспечивающих согласованность данных и состояний между узлами распределённой системы.

Далее рассматриваются существующие подходы к верификации соответствия формальной TLA+ спецификации и программной реализации. После этого формулируется математическая постановка задачи верификации соответствия в терминах системы переходов, трассы исполнения программы и критерия согласованности поведения реализации со спецификацией. Таким образом, исследовательская часть задаёт теоретическую основу как для проектирования системы распределённых вычислений, так и для последующей разработки средств верификации её соответствия формальной модели.

1 Модель распределённой системы

В данной работе распределённая система рассматривается как совокупность независимых вычислительных узлов, взаимодействующих посредством обмена сообщениями по каналам связи и совместно реализующих единое функциональное поведение. В соответствии с определением Э. Таненбаума[1]:

«Распределенная вычислительная система (РВС) – это набор соединенных каналами связи независимых компьютеров, которые с точки зрения пользователя некоторого программного обеспечения выглядят единым целым».

Такое представление соответствует рассматриваемой в работе системе распределённых вычислений, в которой множество узлов совместно обрабатывает задачи, обменивается служебной информацией и поддерживает согласованное состояние. Каждый узел функционирует как самостоятельный вычислительный компонент, однако итоговое поведение системы определяется не отдельными узлами, а их согласованным взаимодействием.

В разрабатываемой системе каждый узел обладает собственным локальным состоянием, выполняет предусмотренные алгоритмом действия и может отправлять сообщения другим узлам. Общее состояние системы формируется совокупностью состояний всех узлов, а также результатами их взаимодействия при передаче команд, распределении ролей и обработке вычислительных задач. Внешние пользователи или клиентские процессы могут направлять системе запросы, инициирующие изменение её состояния и запуск вычислений.

Для дальнейшего анализа важны три группы предположений: свойства каналов связи, временная модель системы и допустимые типы отказов. Именно эти предположения определяют применимость алгоритмов консенсуса и границы корректного функционирования распределённой системы.

1.1 Модель каналов связи

В распределённых системах обмен сообщениями не является абсолютно надёжным: сообщения могут задерживаться, теряться, приходить повторно или наблюдаться в различном порядке. В качестве базовой абстракции часто используется канал с допустимыми потерями (fair-loss link) [2]. Для него характерны следующие свойства:

- если корректный процесс отправляет сообщение бесконечно много раз, то корректный получатель в конечном итоге получит его бесконечное число раз;
- одно отправленное сообщение не может быть доставлено бесконечное число раз;
- канал не создаёт новых сообщений.

Данная абстракция полезна как исходная модель ненадёжного сетевого взаимодействия, однако её недостаточно для построения практических алгоритмов консенсуса. Поэтому в распределённых протоколах обычно используются дополнительные механизмы повышения надёжности: повторные передачи, подтверждения доставки, идентификаторы сообщений и средства устранения дубликатов.

С учётом этих механизмов вводится более сильная абстракция — совершенный канал связи (perfect link), обеспечивающий следующие гарантии [2]:

- каждое сообщение, отправленное корректным процессом корректному получателю, в конечном итоге будет доставлено;
- сообщения между двумя процессами доставляются без нарушения порядка;
- сообщение не доставляется более одного раза;
- канал не создаёт сообщений, которые не были отправлены.

При этом даже совершенный канал не устраняет всех проблем распределённого взаимодействия. В реальной системе возможны временная недоступность связи, длительные задержки и разделение сети, при котором часть узлов на некоторый промежуток времени не может обмениваться сообщениями с остальными. Следовательно, при построении модели распределённой системы необходимо учитывать не только свойства отдельного канала, но и возможность нарушения связности сети в целом.

В рамках настоящей работы каналы связи рассматриваются на уровне абстракции, достаточном для анализа алгоритма консенсуса и поведения распределённой системы вычислений. Существенным является не моделирование низкоуровневых сетевых деталей, а фиксация того факта, что обмен сообщениями может сопровождаться задержками, временной недоступностью и повторными передачами, что непосредственно влияет на выбор лидера, репликацию состояния и согласованность узлов системы.

1.2 Консенсус в распределённой системе

Центральной задачей распределённой системы, в которой несколько узлов совместно поддерживают единое состояние, является достижение консенсуса. Под консенсусом понимается согласование общего решения между множеством процессов, каждый из которых действует независимо и обладает лишь частичной информацией о глобальном состоянии системы.

Корректный алгоритм консенсуса должен обеспечивать выполнение следующих свойств [3]:

- *согласованность* — все корректные процессы принимают одно и то же решение;
- *действительность* — принятое значение должно быть допустимым и происходить из множества предложенных значений;
- *окончательность* — процесс, принявший решение, не изменяет его в дальнейшем, а алгоритм должен приводить систему к устойчивому результату.

Для алгоритмов консенсуса эти свойства имеют фундаментальное значение, поскольку именно они обеспечивают согласованность реплик, корректность выбора лидера и единый порядок применения команд. В настоящей работе консенсус рассматривается как основа согласования состояния узлов распределённой системы вычислений, обеспечивающая единый порядок применения команд, корректность выбора лидера и устойчивость системы к частичным отказам.

1.3 Временная модель и ограничения достижимости консенсуса

Существенное влияние на возможность достижения консенсуса оказывает временная модель распределённой системы. В полностью асинхронной системе отсутствуют гарантированные верхние оценки времени доставки сообщений и времени выполнения шагов процесса. В таких условиях узел не может определить, отказал ли другой узел, либо его ответ лишь задерживается.

Фундаментальным результатом для таких систем является теорема Фишера–Линча–Патерсона о невозможности детерминированного консенсуса в полностью асинхронной среде даже при наличии всего одного отказа процесса [4]. Этот результат показывает, что алгоритм, гарантирующий достижение консенсуса за конечное время при любых асинхронных задержках, в общем случае невозможен.

На практике распределённые системы обычно рассматриваются не как полностью асинхронные, а как частично синхронные. Для такой модели предполагается, что после некоторого неизвестного момента времени задержки сообщений и скорости выполнения процессов оказываются ограниченными сверху, либо такие ограничения выполняются в большинстве типичных сценариев [2]. Именно это предположение лежит в основе большинства практически применимых невизантийских алгоритмов консенсуса.

Следовательно, при анализе распределённой системы необходимо учитывать наличие временного аспекта, однако не требуется вводить полностью синхронную модель с жёстко фиксированными задержками. Для рассматриваемого класса систем достаточно предположения о частичной синхронности, при котором алгоритм консенсуса сохраняет работоспособность в типичных сценариях.

1.4 Модели отказов

Неотъемлемой характеристикой распределённой системы является модель отказов, определяющая, какие нарушения поведения узлов и каналов связи допускаются в рамках анализа. От выбора модели отказов зависит как проектирование алгоритма консенсуса, так и архитектура самой распределённой системы.

В данной работе рассматриваются невизантийские модели отказов. Это означает, что узлы системы либо работают в соответствии с алгоритмом, либо временно перестают выполнять требуемые действия, но не пытаются произвольно искажать данные и не действуют злонамеренно.

Основными типами отказов, существенными для настоящей работы, являются следующие:

- *аварийное завершение* — процесс прекращает выполнение шагов алгоритма и перестаёт участвовать в обмене сообщениями;
- *восстановление после сбоя* — после отказа процесс может быть перезапущен и синхронизирован с текущим состоянием системы, если алгоритм предусматривает сохранение устойчивого состояния [5];
- *пропуск действий* — отдельные шаги алгоритма или результаты этих шагов могут не наблюдаться другими узлами из-за потерь сообщений, перегрузки или временной недоступности связи.

Византийские ошибки в рамках данной работы не рассматриваются. Это означает, что не моделируются ситуации, в которых узел произвольно искажает сообщения, распространяет противоречивую информацию разным участникам или преднамеренно нарушает правила протокола.

Для невизантийских алгоритмов консенсуса основным средством повышения отказоустойчивости является избыточность. За счёт репликации состояния и принятия решений на основе большинства система способна сохранять корректность при отказе части узлов [6].

1.5 Принятые в работе допущения

На основании рассмотренных свойств распределённых систем в данной работе принимаются следующие допущения:

- система состоит из конечного множества взаимодействующих узлов, каждый из которых обладает локальным состоянием и может обмениваться сообщениями с другими узлами;
- узлы совместно поддерживают согласованное состояние на основе невизантийского алгоритма консенсуса;
- в системе допускаются задержки сообщений, временная недоступность связи и разделение сети;
- временная модель считается частично синхронной;

- существенными типами отказов являются аварийное завершение, временная недоступность узла и пропуск отдельных действий или сообщений;
- византийские отказы не рассматриваются;
- модель системы должна быть достаточна для описания поведения выбранного алгоритма консенсуса и основных сценариев функционирования распределённой системы вычислений.

Таким образом, используемая в работе модель распределённой системы ориентирована не на максимально полное воспроизведение всех аспектов сетевой среды, а на выделение тех свойств, которые существенны для анализа алгоритма консенсуса и проектирования отказоустойчивой системы распределённых вычислений.

2 Обзор и анализ существующих алгоритмов консенсуса

Алгоритмы консенсуса предназначены для обеспечения согласованности данных и состояний между узлами распределённой системы в условиях сетевых задержек, частичных отказов и отсутствия общего глобального состояния. Для системы распределённых вычислений это особенно важно, поскольку ошибка в согласовании приводит не только к расхождению журналов реплик, но и к потере задач, повторному выполнению команд, некорректному назначению вычислительных ролей и нарушению целостности результатов вычислений.

В рамках невизантийской модели отказов наибольшее практическое значение получили алгоритмы семейства Paxos и алгоритм Raft. Оба подхода обеспечивают согласованное принятие решений на основе кворума, однако различаются по структуре протокола, организации управления кластером и удобству использования в системах с реплицируемым состоянием.

2.1 Алгоритм Paxos

Алгоритм Paxos является одним из наиболее известных классических алгоритмов достижения консенсуса в распределённых системах. Он был предложен Л. Лэмпортом в работе «The Part-Time Parliament» [7], а затем изложен в более прикладной форме в статье «Paxos Made Simple» [8].

В классическом Paxos обычно выделяют три логические роли:

- предлагающие узлы, формирующие предложения;
- принимающие узлы, голосующие за принятие предложений;
- узлы, фиксирующие итоговое выбранное значение.

На практике эти роли могут совмещаться в одном процессе, однако их разделение удобно для описания логики алгоритма. Каждое предложение задаётся парой (n, v), где n — уникальный номер предложения, а v — предлагаемое значение. Корректность алгоритма обеспечивается тем, что номера предложений упорядочены и позволяют однозначно различать более ранние и более поздние попытки согласования.

Классический Paxos состоит из двух фаз. На первой фазе предлагающий узел отправляет кворуму принимающих узлов сообщение подготовки $Prepare(n)$ и пытается получить от них обещание не принимать предложения с меньшими номерами. На второй фазе, получив кворум ответов, он отправляет сообщение принятия $Accept(n, v)$. Значение считается выбранным, когда его принял кворум принимающих узлов [8].

Ключевое свойство Paxos основано на пересечении кворумов: любые два кворума имеют хотя бы один общий узел. Благодаря этому, если некоторое значение уже было выбрано, информация о нём неизбежно будет учтена в последующих раундах, и противоречащее значение не сможет быть выбрано позже. Именно это обеспечивает безопасность алгоритма.

Обобщённая структура раунда Paxos приведена на рис. 1. Из схемы видно, что в базовом варианте для согласования даже одного значения требуется последовательное выполнение двух фаз с обменом сообщениями между узлами кворума.

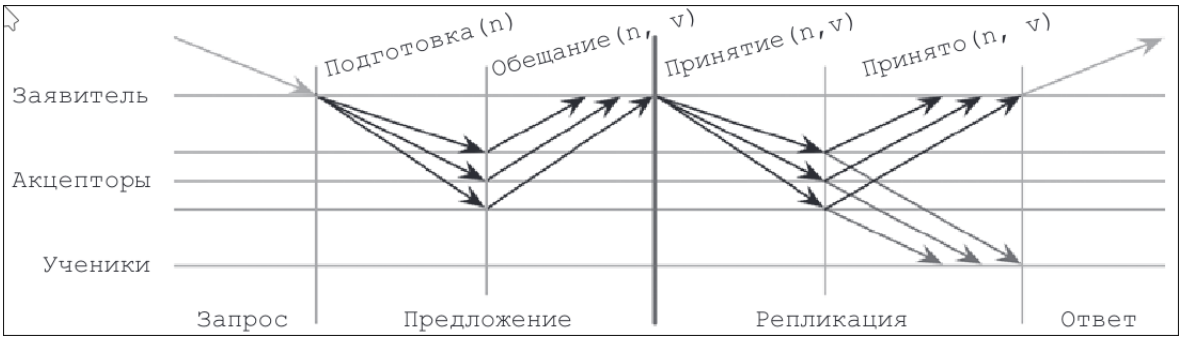

Рисунок 1 – Структура раунда алгоритма Paxos

С точки зрения построения системы распределённых вычислений Paxos обладает важным достоинством: он обеспечивает строгие гарантии согласованности даже в условиях отказа части узлов. Однако его базовый вариант сложен в реализации по нескольким причинам.

Во-первых, согласование каждого нового значения требует повторного выполнения двух фаз протокола. Это увеличивает число сетевых обменов и создаёт дополнительные задержки при последовательной обработке задач, что неудобно для системы, в которой необходимо поддерживать общий журнал состояний, команд и результатов.

Во-вторых, в классическом Paxos отсутствует явно выделенная и постоянно действующая схема управления журналом. Алгоритм корректно решает задачу выбора одного значения, однако переход от согласования отдельного значения к согласованию последовательности операций требует дополнительных механизмов: организации общего журнала, поддержания порядка записей, восстановления после сбоев и согласования действий узлов при повторных раундах.

В-третьих, корректная реализация Paxos требует аккуратного обращения с номерами предложений, кворумами и ранее принятыми значениями. Узел, инициирующий новый раунд, обязан учитывать ответы принимающих узлов и, если некоторое значение уже было принято ранее, продолжать именно его, а не произвольно предлагать новое. Ошибка в этой логике приводит к нарушению безопасности, поэтому реализация требует строгого соблюдения условий перехода между фазами и корректной обработки устаревших, задержанных и повторно пришедших сообщений.

В-четвёртых, состояние принимающих узлов должно сохраняться таким образом, чтобы после сбоя не терялись сведения о данных обещаниях и ранее принятых предложениях. Иначе после перезапуска узел может начать вести себя так, как будто часть истории согласования отсутствовала, что недопустимо для корректной работы протокола.

Таким образом, сложность Paxos в реализации связана не столько с отдельными шагами алгоритма, сколько с тем, что для построения полноценной системы распределённых вычислений на его основе необходимо дополнительно решать задачи управления журналом, выбора ведущего узла, сохранения устойчивого состояния и корректной обработки конкурирующих раундов согласования.

2.2 Алгоритм Multi-Paxos

Для уменьшения накладных расходов классического Paxos используется его расширение, обычно называемое многократным вариантом Paxos (Multi-Paxos). В этом случае после стабилизации ведущего узла подготовительная фаза не повторяется для каждой новой команды. Это позволяет использовать уже достигнутое лидерство для последовательного согласования ряда записей журнала.

Для системы распределённых вычислений такое расширение существенно, поскольку позволяет перейти от согласования отдельного значения к согласованию последовательности операций: регистрации задачи, назначения исполнителя, фиксации промежуточного состояния и записи итогового результата. Тем самым многократный Paxos делает алгоритм более пригодным для построения реплицируемого журнала.

Тем не менее даже в таком виде алгоритм сохраняет высокую концептуальную и инженерную сложность. Хотя многократный вариант Paxos устраняет необходимость повторять подготовительную фазу для каждой записи, его корректная реализация всё равно требует поддерживать устойчивое состояние принимающих узлов, обрабатывать смену ведущего узла, учитывать возможное пересечение и конкуренцию раундов, а также обеспечивать непротиворечивое восстановление журнала после сбоев.

Кроме того, многократный Paxos обычно описывается не как один жёстко фиксированный протокол, а как семейство близких практических схем, в которых разработчику необходимо самостоятельно принимать решения о порядке восстановления, обработке пропусков в журнале, моменте фиксации записей и взаимодействии с клиентскими запросами. Поэтому многократный Paxos является эффективным, но сравнительно сложным средством построения отказоустойчивой распределённой системы.

2.3 Алгоритм Raft

Алгоритм Raft был предложен Д. Онгаро и Дж. Устерхаутом как более понятная и практически ориентированная альтернатива Paxos [9]. Его основная идея заключается в явной декомпозиции задачи консенсуса на выбор лидера, репликацию журнала и обеспечение свойств безопасности.

В Raft каждый узел находится в одном из трёх состояний:

- последователь (follower);
- кандидат (candidate);
- лидер (leader).

При запуске узлы находятся в состоянии последователя. Если последователь в течение тайм-аута выборов не получает сообщений от действующего лидера, он увеличивает номер терма (term), переходит в состояние кандидата и рассылает запросы голосования другим узлам. Если кандидат получает голоса большинства, он становится лидером текущего терма [9].

После избрания лидер начинает отправлять последователям регулярные служебные сообщения, подтверждающие его активность. Эти сообщения одновременно могут использоваться для репликации новых записей журнала. Каждая запись журнала имеет индекс и номер терма. Для добавления записей лидер передаёт последователю номер текущего терма, индекс и терм предыдущей записи, а также одну или несколько новых записей. Последователь принимает новые записи только в том случае, если его журнал согласуется с указанной предыдущей записью. При обнаружении расхождения лидер повторяет попытку с более ранней позицией журнала.

Запись считается зафиксированной, когда она сохранена на большинстве узлов. После этого лидер может считать её согласованной и применять соответствующую команду к состоянию системы. Существенно, что новым лидером может стать только узел с достаточно актуальным журналом, поэтому зафиксированные записи не теряются при смене лидера.

Обобщённая структура раунда Raft приведена на рис. 2. По сравнению с Paxos здесь явно выделены выборы лидера, поддержание лидерства и репликация журнала, что делает алгоритм естественным для систем, где требуется единый журнал состояний и команд.

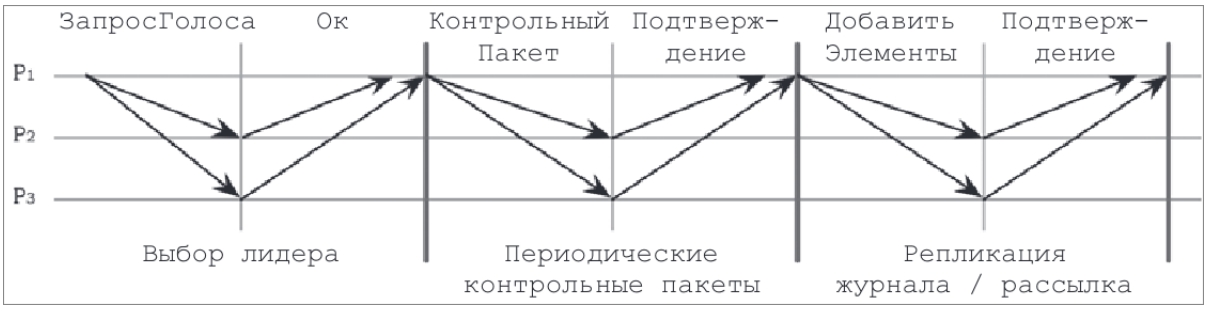

Рисунок 2 – Структура раунда алгоритма Raft

Для системы распределённых вычислений Raft имеет ряд существенных достоинств. Во-первых, он изначально ориентирован на ведение реплицируемого журнала, в который естественным образом отображаются события жизненного цикла вычислительной задачи. Во-вторых, наличие единственного лидера упрощает координацию кластера: через лидера удобно проводить регистрацию задач, выдачу команд рабочим узлам, фиксацию результатов и управление восстановлением после сбоев. В-третьих, правила репликации и фиксации записей в Raft выражены явно, что упрощает реализацию и сопровождение системы.

2.4 Сравнительный анализ

Алгоритмы Paxos и Raft обеспечивают консенсус в распределённой системе с невизантийскими отказами и опираются на идею кворума. Оба алгоритма допускают отказ части узлов и сохраняют безопасность даже при наличии сетевых задержек и временной недоступности отдельных процессов. Следовательно, оба подхода в принципе могут использоваться как основа отказоустойчивой системы распределённых вычислений.

Однако их пригодность для такой системы различается. В Paxos первичной единицей согласования является отдельное значение, а переход к последовательной репликации множества значений требует дополнительных расширений. В Raft реплицируемый журнал является центральной частью алгоритма с самого начала. Для распределённой системы вычислений это особенно важно, поскольку её состояние обычно представляется именно в виде последовательности согласованных операций над журналом: приём задачи, изменение её статуса, назначение исполнителя, подтверждение завершения, фиксация результата.

Существенно различается и организация управления кластером. В классическом Paxos ведущий узел не является обязательной частью базовой модели, тогда как в Raft лидер выступает центральным координатором. Для распределённой системы вычислений это даёт практическое преимущество: упрощается маршрутизация запросов, централизуется принятие решений о фиксации записей, а поведение кластера становится более предсказуемым при сбоях и восстановлении узлов.

С инженерной точки зрения важным различием является и то, что в Paxos механизм репликации журнала получается как развитие базовой схемы согласования, тогда как в Raft он встроен в алгоритм изначально. По этой причине при реализации Paxos разработчику приходится отдельно увязывать кворумное согласование значений с ведением журнала, выбором ведущего узла, восстановлением после отказов и обработкой конкурирующих раундов. В Raft эти элементы описаны как части единого протокола, что делает поведение системы более прямолинейным и предсказуемым.

Сравнение Paxos и Raft по основным техническим критериям приведено в табл. 1. Из таблицы видно, что Paxos обеспечивает строгую кворумную схему согласования и имеет фундаментальное значение для теории распределённых систем. В то же время Raft лучше соответствует архитектуре системы распределённых вычислений, основанной на реплицируемом журнале и выделенном координаторе.

Итак, Paxos и его вариация представляют собой фундаментальную основу невизантийского консенсуса и обеспечивают строгие гарантии согласованности. Однако для системы распределённых вычислений особенно важны удобство ведения реплицируемого журнала, простота координации узлов, предсказуемость поведения при отказах и ясность структуры протокола. По этим критериям алгоритм Raft лучше соответствует требованиям распределённой вычислительной системы, хотя окончательное обоснование его выбора будет приведено в конструкторской части работы.

Таблица 1 – Сравнение алгоритмов Paxos и Raft

Критерий	**Paxos**	**Raft**
Базовая единица согласования	Одно значение; для последовательности значений обычно требуется многократный вариант	Запись реплицируемого журнала
Организация управления	В базовом варианте ведущий узел не является обязательным элементом; лидерство вводится в практических расширениях	Лидер является центральным элементом алгоритма
Структура раунда	Две фазы: подготовка и принятие	Выборы лидера, подтверждение лидерства, репликация и фиксация записей
Работа с журналом состояния	Журнал не является центральным элементом базовой модели; требуется дополнительная логика его ведения и восстановления	Журнал является основой алгоритма и репликации состояния
Координация кластера	Требует доп. соглашений в практических реализациях	Естественно строится вокруг лидера
Поведение при смене ведущего узла	Корректно, но логика сохранения значения задаётся через кворумы, номера предложений и устойчивое состояние узлов	Корректно, за счёт явных правил актуальности журнала и выборов лидера
Удобство реализации системы распределённых вычислений	Требует дополнительного связывания кворумного согласования с ведением журнала, выбором ведущего узла и восстановлением после сбоев	Более естественно подходит для координации задач, репликации состояния и восстановления после сбоев

3 Теоретические основы системы верификации

В основе разрабатываемой системы верификации лежат методы формальной спецификации и модельной проверки распределённых систем. Формальные методы позволяют описывать систему в виде математической модели, задающей множество допустимых состояний и переходов между ними, а также формулировать инварианты и свойства корректности. Однако проверка самой спецификации не гарантирует корректность конкретной программной реализации. Для сопоставления поведения программы с её формальной моделью используется подход, основанный на проверке трасс исполнения по формальной спецификации.

3.1 Формальная спецификация на языке TLA+

TLA+ представляет собой язык формальных спецификаций, основанный на теории множеств, логике первого порядка и темпоральной логике действий. Он предназначен для описания поведения дискретных систем в терминах состояний и переходов между ними [10].

В TLA+ система задаётся с помощью переменных состояния. Состояние представляет собой присваивание значений этим переменным, а шаг — переход от одного состояния к другому. Поведение системы задаётся как последовательность состояний. Предикат состояния описывает свойства отдельного состояния, а действие — свойства перехода между двумя соседними состояниями. Для различения значений переменных в текущем и следующем состояниях используются обычные и штрихованные обозначения [11].

Типичная структура спецификации имеет вид

$$Spec \triangleq Init \wedge \Box[Next]_{(v_1, v_2, \ldots, v_n)}$$

где $Init$ задаёт множество начальных состояний, $Next$ определяет допустимые переходы системы, а $(v_1, v_2, \ldots, v_n)$ — множество переменных состояния. Темпоральный оператор $\Box$ выражает требование истинности формулы на всём протяжении поведения системы.

Практическая работа со спецификациями TLA+ обычно выполняется с использованием средства проверки моделей TLC. Оно строит пространство достижимых состояний, проверяет инварианты и позволяет обнаруживать контрпримеры, нарушающие заданные свойства системы. В стандартном сценарии TLA+ спецификация используется для проверки свойств самой модели. В рамках настоящей работы спецификация рассматривается также как эталон допустимого поведения, с которым затем сопоставляется наблюдаемое поведение программной реализации.

3.2 Проверка трасс исполнения по формальной модели

Одной из важных практических задач при использовании TLA+ является проверка соответствия программной реализации её формальной спецификации. В идеальном случае хотелось бы доказать, что реализация является уточнением высокоуровневой спецификации, то есть что все возможные поведения программы допустимы спецификацией. Однако на практике такой подход сталкивается с серьёзными проблемами масштабируемости и применим лишь к ограниченному кругу сравнительно небольших систем.

Полная сквозная верификация остаётся крайне трудоёмкой и технически сложной задачей вне зависимости от используемых инструментов. Даже в тех случаях, когда она теоретически осуществима, затраты времени и ресурсов делают её непрактичной для большинства прикладных систем [12].

Поэтому в инженерной практике используется более прагматичный подход, основанный на проверке трасс исполнения по формальной модели [13]. Его идея заключается не в доказательстве того, что *все* возможные поведения реализации соответствуют спецификации, а в проверке того, что *наблюдаемые* поведения не противоречат ей.

Подход включает следующие этапы:

а) программная реализация инструментируется таким образом, чтобы фиксировать существенные события и изменения состояния;
б) во время выполнения системы формируется трасса исполнения;
в) полученная трасса интерпретируется как ограничение на допустимое поведение формальной спецификации;
г) средство модельной проверки определяет, существует ли поведение спецификации, согласованное с наблюдаемой трассой.

Таким образом, задача сводится к проверке воспроизводимости наблюдаемой последовательности переходов в рамках формальной модели. В отличие от классической модельной проверки, здесь не требуется перебирать всё пространство состояний реализации: анализируется лишь конкретная трасса исполнения. Это существенно снижает вычислительную сложность и делает метод применимым к распределённым системам, где трассы могут собираться с нескольких узлов и объединяться для последующего анализа.

Данный подход обладает рядом практических преимуществ:

- относительно низкая стоимость интеграции в существующий процесс разработки;
- возможность использования стандартных средств модельной проверки;
- обнаружение некорректных переходов даже при отсутствии явных отказов выполнения;
- применимость к системам, реализованным на различных языках программирования.

Следует отметить, что такой метод не даёт полной формальной гарантии корректности, поскольку анализируется лишь конечное множество наблюдаемых трасс. Тем не менее при достаточном количестве тестовых запусков вероятность обнаружения ошибок существенно возрастает. Как показывает практика применения формальных методов в индустрии, частичная формальная проверка позволяет выявлять серьёзные ошибки на ранних этапах разработки [14].

3.3 Существующие подходы

Проверка трасс исполнения на соответствие высокоуровневым свойствам или спецификациям имеет длительную историю в области формальных методов. К. Хавелунд предложил верификацию во время выполнения как облегчённый подход, позволяющий проверять, согласуется ли трасса исполнения системы с её высокоуровневой спецификацией [15]. Как правило, такой подход предполагает построение монитора (monitor) на основе спецификации; далее монитор получает события трассы и проверяет их на соответствие требованиям [16].

Ховард и соавторы предложили проверять трассы исполнения непосредственно относительно высокоуровневой спецификации системы [13]. В их работе также было показано, что для такой проверки могут использоваться стандартные средства модельной проверки, в частности ProB и Spin. Однако в данной работе не рассматривались распределённые программы, для которых требуется использование централизованных или распределённых часов для сохранения причинно-следственных связей, а также трассы с неполной информацией.

Тасиран и соавторы одними из первых извлекали и проверяли трассы, полученные из аппаратного симулятора, на соответствие TLA+ спецификации, тем самым продемонстрировав практическую применимость валидации трасс [17]. Распространение TLA+ среди практиков распределённых систем, во многом обусловленное работой Ньюкомба и соавторов [18], а также формализация валидации трасс как проверки уточнения, предложенная Пресслером [19], привели к использованию данного подхода в реальных распределённых системах.

Так, Дэвис и соавторы применили валидацию трасс к MongoDB и обнаружили нетривиальную ошибку реализации [20]. При этом авторы столкнулись с трудностями последовательного журналирования состояния реализации и согласования различной гранулярности атомарности между спецификацией и программой. Ниу и соавторы также использовали проверку трасс для валидации ZooKeeper, подтверждая соответствие реализации спецификации [21]. Аналогично, Ван и соавторы выявили несколько ошибок реализации, воспроизводя поведения TLA+ на инструментированных реализациях [22]. Их работа также демонстрирует сложности, связанные с согласованием различной гранулярности атомарности между спецификацией и кодом. Тем не менее все указанные работы показывают, что сопоставление трасс реализации с высокоуровневыми поведениями TLA+ позволяет находить нетривиальные ошибки в реальных системах [23].

Для более тесного согласования высокоуровневых спецификаций с их практическими реализациями Хэкетт и соавторы [24], а также Фу и соавторы [25] предложили расширения языка PlusCal, который используется как алгоритмическая надстройка над TLA+. Эти расширения направлены, соответственно, на добавление в спецификации достаточного уровня детализации для генерации кода и на генерацию мониторов времени

выполнения. Несмотря на перспективность таких подходов, необходимость введения дополнительных прослоек между спецификацией и реализацией может затруднять их широкое промышленное применение. Кроме того, проекция глобальной машины состояний, которая является распространённым стилем моделирования, на локальные машины состояний отдельных узлов затрудняет проверку глобальных свойств с помощью локальных мониторов.

С рассматриваемым направлением также связаны методы тестирования реализаций по формальным спецификациям, например подходы на основе соответствия входов и выходов [26]. Их общая цель также состоит в повышении уверенности в корректности реализации, однако в таких подходах реализация обычно рассматривается как чёрный ящик, допускающий лишь наблюдения на интерфейсах. Аналогичное замечание относится и к методам активного обучения конечных автоматов [27]. В отличие от этого, при валидации трасс предполагается наличие доступа к исходному коду реализации и возможность его инструментирования, что даёт разработчику больше средств для преодоления расхождений между гранулярностью атомарности спецификации и программы.

Таким образом, существующие подходы к верификации соответствия TLA+ спецификации и исходного кода можно условно разделить на несколько направлений: верификация во время выполнения с использованием мониторов, проверка трасс исполнения по высокоуровневой формальной модели, расширения языков спецификаций для генерации кода и мониторов, а также методы тестирования и обучения моделей по наблюдаемому поведению. Среди них наибольший интерес для настоящей работы представляет именно проверка трасс исполнения по формальной модели, поскольку она сочетает практическую применимость, сравнительно низкую стоимость внедрения и возможность использовать уже существующую TLA+ спецификацию как эталон допустимого поведения реализации.

4 Математическая постановка задачи

В данном разделе формализуется задача проверки того, может ли наблюдаемая трасса исполнения программы быть объяснена допустимым поведением абстрактной модели. Для этого модель задаётся как система переходов, трасса программы — как последовательность наблюдений, а связь между наблюдением и шагом модели — как предикат согласованности. На этой основе вводятся понятия совместимости трассы с моделью, максимального согласованного префикса и показателя качества согласованности.

4.1 Формализация модели и ее поведения

Для формализации модели (спецификации) программы используется понятие *системы переходов* (transition system) [**Keller1976**] с выделенным множеством начальных состояний.

Определение 4.1. Системой переходов называется тройка

$$\mathcal{M} = (X, X_0, R), \tag{1}$$

где

– X — множество состояний;
– $X_0 \subseteq X$ — множество начальных состояний;
– $R \subseteq X \times X$ — отношение переходов.

Элемент $x \in X$ интерпретируется как полное абстрактное описание состояния системы в некоторый момент её выполнения. Условие $(x, y) \in R$ означает, что из состояния x система может перейти в состояние y за один допустимый шаг. Иначе говоря, отношение R задаёт все допустимые одношаговые изменения состояния системы.

Определение 4.2. Конечным поведением системы переходов $\mathcal{M}$ длины $n \in \mathbb{N}_0$ называется последовательность состояний

$$\pi_n = (x_0, x_1, \ldots, x_n) \in X^{n+1}, \tag{2}$$

для которой выполнены условия

$$x_0 \in X_0, \qquad \forall i \in \{0, \ldots, n-1\} : (x_i, x_{i+1}) \in R.$$

Первое условие означает, что поведение начинается из некоторого допустимого начального состояния. Второе условие означает, что каждый следующий элемент последовательности получается из предыдущего допустимым переходом системы. Таким образом, конечное поведение есть путь в ориентированном графе, порождаемом системой переходов, начинающийся в одной из вершин из множества X_0.

Определение 4.3. Для каждого $n \in \mathbb{N}_0$ обозначим через $\mathrm{B}_n(\mathcal{M})$ множество всех конечных поведений системы переходов $\mathcal{M}$ длины n:

$$\mathrm{B}_n(\mathcal{M}) = \left\{(x_0, \ldots, x_n) \in X^{n+1} : x_0 \in X_0,\ \forall i \in \{0, \ldots, n-1\} : (x_i, x_{i+1}) \in R\right\}.$$

Тогда множество всех конечных поведений системы переходов $\mathcal{M}$ обозначается через $\mathrm{B}(\mathcal{M})$ и определяется равенством

$$\mathrm{B}(\mathcal{M}) = \bigcup_{n \in \mathbb{N}_0} \mathrm{B}_n(\mathcal{M}). \tag{3}$$

Множество $\mathrm{B}(\mathcal{M})$ содержит все конечные пути в системе переходов, начинающиеся из допустимых начальных состояний. Тем самым оно полностью определяет допустимое наблюдаемое поведение модели на конечных префиксах.

Если некоторое исполнение реализации не может быть согласовано ни с одним элементом множества $\mathrm{B}(\mathcal{M})$, то это означает, что наблюдаемое поведение не принадлежит множеству допустимых поведений, задаваемых абстрактной моделью. Следовательно, в этом случае фиксируется расхождение между реализацией и спецификацией.

4.2 Формализация трассы программы

Реальная программа в процессе выполнения порождает наблюдаемую трассу. В рассматриваемой постановке предполагается, что трасса не содержит отдельного наблюдения начального состояния системы. Начальное состояние выбирается из множества начальных состояний модели, а трасса фиксирует только наблюдения, относящиеся к последующим шагам выполнения программы.

При этом в рамках рассматриваемой постановки предполагается, что каждой записи трассы соответствует ровно один переход поведения модели. Иначе говоря, трасса и поведение модели сопоставляются *пошагово*.

Определение 4.4. Пусть Σ — множество всех возможных наблюдений. Тогда наблюдаемая трасса длины $m \in \mathbb{N}_0$ задаётся последовательностью

$$\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m. \tag{4}$$

Каждый элемент σ_i интерпретируется как наблюдение, относящееся к i-му шагу выполнения программы. Таким образом, поведение модели задаётся последовательностью состояний $(x_0, x_1, \ldots, x_m)$, тогда как наблюдаемая трасса задаётся последовательностью наблюдений $(\sigma_1, \sigma_2, \ldots, \sigma_m)$, в которой каждый элемент σ_i относится к переходу из состояния x_{i-1} в x_i.

Определение 4.5. Для каждого $k \in \{0, \ldots, m\}$ префиксом трассы длины k называется последовательность

$$\tau^{(k)} = (\sigma_1, \sigma_2, \ldots, \sigma_k) \in \Sigma^k,$$

где при $k = 0$ понимается пустая последовательность.

4.3 Согласованность трассы с моделью

После введения понятий модели и трассы можно строго определить, что означает их согласованность.

В общем случае трасса программы представляет собой последовательность неполных изменений состояния системы: обычно она содержит лишь частичную информацию о каждом шаге выполнения, существенную для верификации, например сведения о произошедшем событии и об изменениях отдельных компонент состояния. Поэтому необходимо формально задать, когда переход считается согласованным с данным наблюдением.

Определение 4.6. Предикатом согласованности наблюдаемого шага называется отображение

$$C : \Sigma \times X \times X \to \{0, 1\}. \tag{5}$$

Значение $C(\sigma, x, y) = 1$ означает, что наблюдение σ согласованно с переходом из состояния x в состояние y.

Предикат C задаёт способ интерпретации трассы в терминах абстрактной модели. Его конкретный вид определяется составом данных, записываемых в трассу. Например, если наблюдение фиксирует имя произошедшего события и значения некоторых изменяемых компонент состояния, то согласованность означает, что рассматриваемый переход принадлежит классу переходов, соответствующему данному событию, и приводит к зафиксированным изменениям состояния. Если же наблюдение фиксирует значения лишь некоторых компонент следующего состояния, то согласованность означает совпадение именно этих компонент, тогда как остальные компоненты могут выбираться произвольно при условии допустимости перехода.

Определение 4.7. Пусть дан префикс трассы $\tau^{(k)} = (\sigma_1, \ldots, \sigma_k)$, где $k \in \{0, \ldots, m\}$. Тогда поведение модели

$$\pi_k = (x_0, x_1, \ldots, x_k) \in X^{k+1}$$

называется *согласованным с префиксом трассы* $\tau^{(k)}$, если выполнены условия

$$x_0 \in X_0, \qquad \forall i \in \{1, \ldots, k\} : C(\sigma_i, x_{i-1}, x_i) = 1.$$

В рамках данной постановки не рассматриваются сопоставления, при которых одному элементу трассы соответствует несколько переходов модели или, наоборот, одному переходу модели соответствует несколько элементов трассы. Такое ограничение соответствует используемой схеме проверки, в которой каждая запись трассы обрабатывается за один шаг ограниченной спецификации.

Определение 4.8. Для каждого $k \in \{0, \ldots, m\}$ обозначим через $\mathrm{M}_k(\tau)$ множество всех поведений длины k, согласованных с префиксом трассы $\tau^{(k)}$:

$$\mathrm{M}_k(\tau) = \left\{(x_0, \ldots, x_k) \in X^{k+1} : x_0 \in X_0,\ \forall i \in \{1, \ldots, k\} : C(\sigma_i, x_{i-1}, x_i) = 1\right\}. \tag{6}$$

Определение 4.9. Префикс трассы $\tau^{(k)}$, где $k \in \{0, \ldots, m\}$, называется *совместимым* с моделью $\mathcal{M}$, если

$$\mathrm{B}_k(\mathcal{M}) \cap \mathrm{M}_k(\tau) \neq \varnothing. \tag{7}$$

Иными словами, совместимость префикса трассы с моделью означает существование хотя бы одного поведения длины k, которое допускается моделью и одновременно объясняет первые k наблюдений трассы.

В частности, вся трасса $\tau = (\sigma_1, \ldots, \sigma_m)$ совместима с моделью $\mathcal{M}$ тогда и только тогда, когда её префикс полной длины m совместим с моделью, то есть выполнено условие

$$\mathrm{B}_m(\mathcal{M}) \cap \mathrm{M}_m(\tau) \neq \varnothing. \tag{8}$$

4.4 Показатель качества согласованности

Для введения количественной оценки согласованности трассы с моделью рассмотрим множество всех допустимых длин согласованных префиксов.

Определение 4.10. Множеством допустимых длин согласованных префиксов называется множество

$$K(\mathcal{M}, \tau) = \{n \in \{0, \ldots, m\} : \mathrm{B}_n(\mathcal{M}) \cap \mathrm{M}_n(\tau) \neq \varnothing\}. \tag{9}$$

Иначе говоря, число n принадлежит множеству $K(\mathcal{M}, \tau)$ тогда и только тогда, когда существует допустимое поведение модели длины n, согласованное с префиксом трассы длины k.

Определение 4.11. Длиной максимального согласованного префикса трассы называется величина

$$k^*(\mathcal{M}, \tau) = \max K(\mathcal{M}, \tau). \tag{10}$$

Величина $k^*(\mathcal{M}, \tau)$ показывает, сколько первых наблюдений трассы можно согласовать с моделью. Если $k^*(\mathcal{M}, \tau) = m$, то вся трасса согласована с моделью. Если же $k^*(\mathcal{M}, \tau) < m$, то согласованным является только собственный префикс трассы.

Определение 4.12. Показателем качества согласованности трассы с моделью называется функция

$$q(k) = \frac{k}{m}, \qquad k \in \{0, \ldots, m\}, \qquad m \geq 1. \tag{11}$$

Тогда оптимальное значение показателя качества определяется равенством

$$Q(\mathcal{M}, \tau) = q(k^*(\mathcal{M}, \tau)) = \frac{k^*(\mathcal{M}, \tau)}{m}. \tag{12}$$

4.5 Итоговая математическая постановка задачи

Пусть $\mathcal{M} = (X, X_0, R)$ — система переходов, где X — множество состояний, $X_0 \subseteq X$ — множество начальных состояний, $R \subseteq X \times X$ — отношение переходов (см. определение 4.1).

Пусть $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m$ — трасса программы, где Σ — множество наблюдений, а σ_i представляет собой наблюдение изменения состояния при переходе из $i-1$ в i (см. определение 4.4).

Пусть $C : \Sigma \times X \times X \to \{0, 1\}$ — предикат согласованности шага, в котором значение $C(\sigma, x, y) = 1$ означает, что наблюдение σ согласованно с переходом из состояния x в состояние y (см. определение 4.6).

Пусть $K(\mathcal{M}, \tau) = \{n \in \mathbb{N}_0 : n \leq m,\ \mathrm{B}_n(\mathcal{M}) \cap \mathrm{M}_n(\tau) \neq \varnothing\}$ — множество допустимых длин согласованных префиксов трассы, где $\mathrm{B}_n(\mathcal{M})$ – множество всех конечных поведений системы переходов $\mathcal{M}$ длины n, а $\mathrm{M}_n(\tau)$ – множество всех поведений длины n, согласованных с префиксом трассы $\tau^{(n)}$ (см. определения 4.3, 4.8, 4.10).

Тогда математическая постановка задачи заключается в нахождении такого значения $k \in \mathbb{N}_0$, для которого показатель качества согласованности максимален при ограничениях

$$k \leq m; \qquad k \in K(\mathcal{M}, \tau).$$

Иначе говоря, требуется решить задачу оптимизации

$$\begin{cases} q(k) \to \max\limits_{k}, \\ k \leq m, \\ k \in K(\mathcal{M}, \tau). \end{cases} \tag{13}$$

При этом исходная задача верификации соответствия трассы и модели является связанной с (13) задачей разрешимости: требуется определить, достигается ли идеальное значение показателя качества, то есть истинно ли условие $q(k^*) = 1$. Это условие эквивалентно равенству $k^* = m$, а следовательно, и условию полной согласованности трассы с моделью: $\mathrm{B}_m(\mathcal{M}) \cap \mathrm{M}_m(\tau) \neq \varnothing$ (по определениям 4.10 и 4.11).

КОНСТРУКТОРСКАЯ ЧАСТЬ

ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

ОРГАНИЗАЦИОННО-ЭКОНОМИЧЕСКАЯ ЧАСТЬ

Экономическое обоснование является важной составляющей инженерного проекта, позволяющей определить трудоёмкость работ, необходимые ресурсы и ожидаемую эффективность внедрения разрабатываемого решения.

Целью организационно-экономического раздела является обоснование целесообразности разработки методики верификации, оценка затрат на её создание, а также анализ потенциальных областей применения и перспектив внедрения. Для достижения цели необходимо решить следующие задачи:

- определить структуру и этапы разработки методики и программного инструмента;
- оценить трудоёмкость выполнения работ;
- определить состав исполнителей и распределение задач;
- рассчитать затраты на реализацию проекта;
- проанализировать область применения и практическую значимость разработанного решения.

5 Описание процесса изготовления продукта

На данном этапе организационно-экономического обоснования определяется состав и последовательность работ, необходимых для разработки методики верификации соответствия TLA+ спецификации и исходного кода, а также программного инструмента, реализующего предложенный подход.

Разработка проекта включает ряд этапов, начиная с анализа предметной области и заканчивая подготовкой документации. Декомпозиция проекта позволяет определить структуру работ, оценить их трудоёмкость и сформировать основу для дальнейшего расчёта затрат.

Таблица 2 – Этапы разработки проекта

Этап	Наименование этапа
1	Анализ предметной области и формирование требований
2	Проектирование архитектуры системы
3	Разработка механизма трассировки
4	Разработка системы обработки трасс в TLA+
5	Тестирование и отладка
6	Подготовка документации

Перечень событий и работ проекта приведён в таблице 3.

Оценка трудоёмкости выполняется на основе экспертных оценок. Для каждой работы задаются минимальное и максимальное значения времени выполнения, после чего рассчитывается ожидаемая трудоёмкость:

$$T = \frac{3T_{\min} + 2T_{\max}}{5} \tag{14}$$

где $T_{\min}$ — минимальная оценка времени выполнения работы, $T_{\max}$ — максимальная оценка.

Результаты расчёта трудоёмкости приведены в таблице 4.

Таким образом, суммарная трудоёмкость разработки проекта составляет 436 человеко-часов, что соответствует 54.5 человеко-дням. Наиболее трудоёмким этапом является разработка механизма трассировки, поскольку именно на этом этапе реализуется библиотека инструментирования исходного кода, обеспечивающая фиксацию событий исполнения и изменений переменных модели.

Для более детального анализа структуры трудоёмкости программных работ используется разложение общей трудоёмкости каждой работы на составляющие. Суммарные трудозатраты на выполнение i-й работы представляются в виде:

$$q_i = q_i^{prog} + q_i^{al} + q_i^{test} + q_i^{doc}, \tag{15}$$

где q_i^{prog} — затраты труда на непосредственную реализацию программного решения; q_i^{al} — затраты труда на алгоритмизацию; q_i^{test} — затраты на тестирование и исправление ошибок; q_i^{doc} — затраты на подготовку документации.

Затраты на алгоритмизацию определяются через коэффициент n_a:

$$q_i^{al} = n_a \cdot q_i^{prog}. \tag{16}$$

Затраты на тестирование и исправление ошибок определяются по формуле:

$$q_i^{test} = q_i^{prog} \cdot (n_t + n_{cor}), \tag{17}$$

где n_t — коэффициент тестирования, n_{cor} — коэффициент внесения исправлений.

Затраты на подготовку документации:

$$q_i^{doc} = n_d \cdot q_i^{prog}. \tag{18}$$

Тогда суммарная трудоёмкость может быть выражена как:

$$q_i = q_i^{prog}(1 + n_a + n_t + n_{cor} + n_d). \tag{19}$$

Примем значения коэффициентов, рекомендуемые методикой:

$$n_a = 0.3, \quad n_t = 0.3, \quad n_{cor} = 0.3, \quad n_d = 0.35.$$

Тогда:

$$q_i = q_i^{prog}(1 + n_a + n_t + n_{cor} + n_d) = q_i^{prog} \cdot 2.25.$$

Отсюда:

$$q_i^{prog} = \frac{q_i}{2.25}. \tag{20}$$

Затраты на отдельные составляющие определяются как:

$$q_i^{al} = 0.3q_i^{prog}, \quad q_i^{test} = 0.6q_i^{prog}, \quad q_i^{doc} = 0.35q_i^{prog}.$$

Рассмотрим пример расчёта структуры трудоёмкости для этапа 1 «Анализ предметной области и формирование требований».

Согласно таблице 4, суммарная трудоёмкость этапа составляет:

$$q_1 = 32 + 40 + 24 = 96 \text{ чел.-ч.}$$

Определим трудозатраты на программную реализацию:

$$q_1^{prog} = \frac{q_1}{2.25} = \frac{96}{2.25} \approx 42.67 \text{ чел.-ч.}$$

Тогда остальные составляющие равны:

$$q_1^{al} = 0.3 \cdot 42.67 \approx 12.80 \text{ чел.-ч,}$$

$$q_1^{test} = 0.6 \cdot 42.67 \approx 25.60 \text{ чел.-ч,}$$

$$q_1^{doc} = 0.35 \cdot 42.67 \approx 14.93 \text{ чел.-ч.}$$

Аналогичным образом выполняются расчёты для остальных этапов. Результаты приведены в таблице 5.

Таблица 3 – Перечень событий и работ проекта

№ события или работы	Событие	Работа	чел.-часы	чел.-дни
1	Начало работ	–	0	0
2	Этап 1	Анализ предметной области и постановка задачи верификации	32	4
3		Исследование языка TLA+, TLC и существующих подходов к trace-checking	40	5
4		Формирование функциональных и нефункциональных требований к системе	24	3
5	Этап 2	Проектирование архитектуры программного инструмента	40	5
6		Разработка структуры модулей и интерфейсов взаимодействия	24	3
7		Выбор технологий реализации и форматов хранения трасс	20	2.5
8	Этап 3	Разработка библиотеки инструментирования исходного кода	112	14
9	Этап 4	Разработка подсистемы агрегации трасс в единую трассу	24	3
10		Разработка подсистемы обработки трасс в TLA+ и сравнение со спецификацией	56	7
11	Этап 5	Тестирование, отладка и исправление выявленных ошибок	44	5.5
12	Этап 6	Подготовка технической документации и описания методики	20	2.5
13	Окончание работ	–	0	0

Таблица 4 – Трудоёмкость работ проекта

Этап	№	Содержание работы	$T_{\min}$	$T_{\max}$	T	чел.-дни	Исполнитель
1	1	Анализ предметной области и постановка задачи верификации	28	38	32	4	Аналитик
1	2	Исследование языка TLA+, TLC и существующих подходов к trace-checking	36	46	40	5	Аналитик
1	3	Формирование функциональных и нефункциональных требований к системе	20	30	24	3	Аналитик
2	4	Проектирование архитектуры программного инструмента	36	46	40	5	Архитектор
2	5	Разработка структуры модулей и интерфейсов взаимодействия	20	30	24	3	Архитектор
2	6	Выбор технологий реализации и форматов хранения трасс	16	26	20	2.5	Архитектор
3	7	Разработка библиотеки инструментирования исходного кода	104	124	112	14	Разработчик
4	8	Разработка подсистемы агрегации трасс в единую трассу	20	30	24	3	Разработчик
4	9	Разработка подсистемы обработки трасс в TLA+ и сравнение со спецификацией	52	62	56	7	Разработчик
5	10	Тестирование, отладка и исправление выявленных ошибок	40	50	44	5.5	Разработчик
6	11	Подготовка технической документации и описания методики	16	26	20	2.5	Аналитик
				Итого	436	54.5	

Таблица 5 – Структура трудоёмкости работ по этапам проекта

Этап	Наименование этапа	q_i, чел.-ч	q_i^{prog}	q_i^{al}	q_i^{test}	q_i^{doc}
1	Анализ предметной области и формирование требований	96	42.67	12.80	25.60	14.93
2	Проектирование архитектуры системы	84	37.33	11.20	22.40	13.07
3	Разработка механизма трассировки	112	49.78	14.93	29.87	17.42
4	Разработка системы обработки трасс в TLA+	80	35.56	10.67	21.33	12.44
5	Тестирование и отладка	44	19.56	5.87	11.73	6.84
6	Подготовка документации	20	8.89	2.67	5.33	3.11
	Итого	436	193.79	58.14	116.26	67.81

6 Диаграмма Ганта и сетевой график изготовления продукта

Для представления последовательности выполнения работ по разработке методики верификации соответствия TLA+ спецификации и исходного кода используется диаграмма Ганта. Она позволяет наглядно отобразить продолжительность отдельных работ, временные зависимости между ними, а также распределение задач между исполнителями проекта.

При построении календарного плана использовались результаты декомпозиции технологического процесса, полученные в предыдущем разделе. В отличие от полностью последовательной схемы выполнения работ, в рассматриваемом проекте часть задач выполняется параллельно. После завершения этапа проектирования начинается паралельная разработка необходимых частей проекта. Это позволяет сократить общую календарную длительность проекта при сохранении рассчитанной трудоёмкости.

Основные события и работы проекта, необходимые для построения линейного и сетевого графиков, приведены в таблице 6.

На основании данных таблицы 6 в среде Microsoft Project была построена диаграмма Ганта (см. рис. 3), отражающая календарную последовательность выполнения задач, их длительность и логические связи между ними. Диаграмма показывает, что работы по анализу предметной области, исследованию существующих подходов и формированию требований выполняются последовательно, поскольку каждая следующая задача опирается на результаты предыдущей.

После завершения этапа анализа требований начинается проектирование архитектуры, затем определяются взаимодействие модулей и используемые технологии. Наиболее длительной работой проекта является разработка библиотеки трассировки, поскольку именно на этом этапе реализуется программная основа предлагаемой методики. После этого выполняются агрегация трасс, их обработка в TLA+, тестирование и подготовка документации.

Таблица 6 – Основные события и работы проекта

№	Событие	Работа	Код	Тип	Часы	Дни
0	Начало работ	–	0–1	–	0	0
1	Анализ предметной области	Анализ предметной области	1–2	ОН	32	4
2		Исследование подходов	2–3	ОН	40	5
3		Формирование требований	3–4	ОН	24	3
4	Проектирование архитектуры	Проектирование архитектуры	4–5	ОН	40	5
5		Разработка взаимодействия модулей	5–6	ОН	24	3
6		Выбор технологий	6–7	ОН	20	3
7	Разработка трассроваки	Разработка библиотеки трассировки	7–8	ОН	112	14
8	Разработка обработки	Подсистема агрегации трасс	8–9	ОН	24	3
9		Обработка трасс в TLA+	9–10	ОН	56	7
10	Тестирование и отладка	Тестирование и отладка	10–11	ОН	44	6
11	Подготовка документации	Подготовка документации	11–12	ОН	20	3
12	Документация подготовлена	Окончание работ	12–13	ОН	0	0

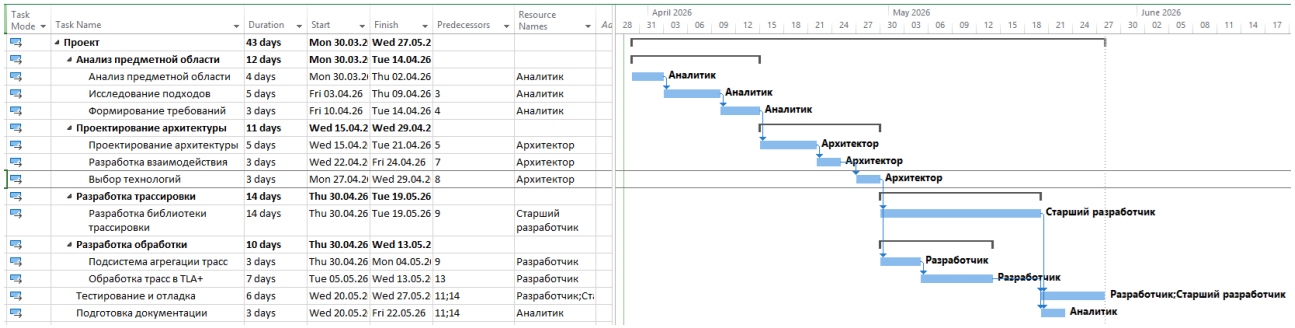

Рисунок 3 – Диаграмма Ганта проекта

Для оценки загрузки исполнителей был построен график использования трудовых ресурсов (см. рис 4). Он позволяет определить, насколько равномерно распределены задачи между участниками проекта, а также выявить возможные перегрузки. Как видно из результатов планирования, основная нагрузка приходится на старшего разработчика и разработчика, что обусловлено высокой

трудоёмкостью этапов реализации библиотеки трассировки, агрегации трасс и их последующей обработки в TLA+. Аналитик активно задействован на начальном и завершающем этапах проекта, а архитектор — на стадии проектирования системы.

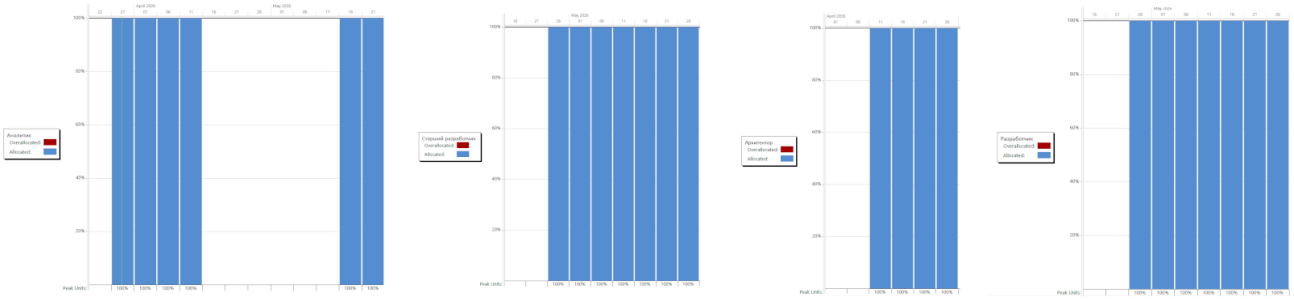

Рисунок 4 – График использования трудовых ресурсов проекта

Для более детального представления распределения работ между исполнителями была сформирована схема распределения ресурсов по задачам (см. рис. 5). Данная схема позволяет установить, какие именно работы выполняются каждым участником проекта и в какие календарные периоды. Такое представление удобно при анализе возможности параллельного выполнения работ и оптимизации календарного плана.

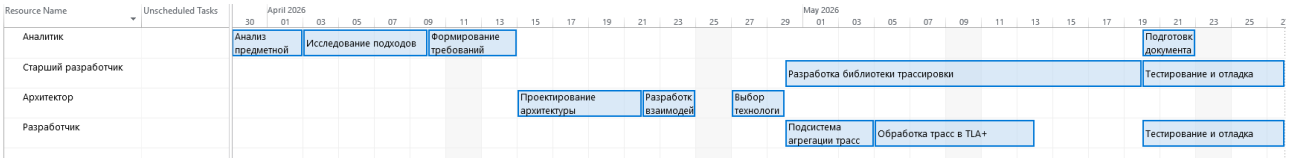

Рисунок 5 – Визуальное представление распределения ресурсов по задачам

Для анализа логической структуры проекта и выделения наиболее ответственных работ был построен сетевой график (см. рис. 6). Сетевой график позволяет определить критический путь проекта, то есть такую последовательность задач, задержка выполнения которой приводит к увеличению общей продолжительности разработки.

В рассматриваемом проекте критический путь проходит через работы, связанные с анализом предметной области, исследованием подходов, формированием требований, проектированием архитектуры, разработкой библиотеки трассировки, агрегацией трасс, обработкой трасс в TLA+, тестированием и подготовкой документации. Следовательно, именно эти работы требуют наиболее тщательного контроля сроков выполнения.

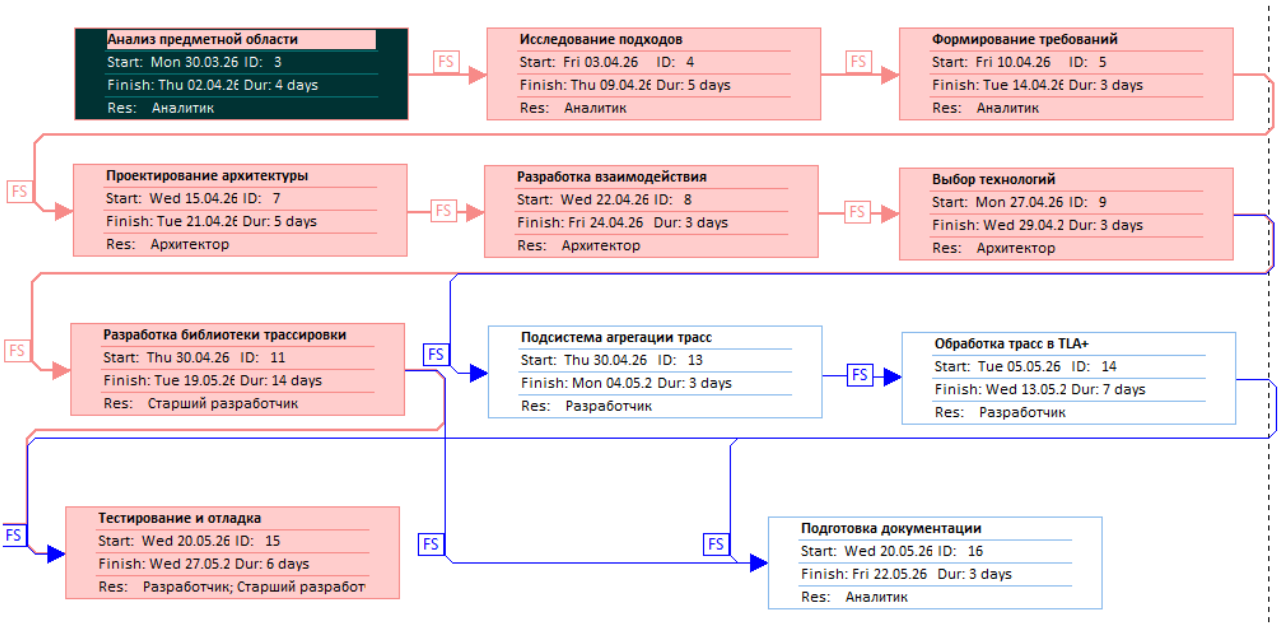

Рисунок 6 – Сетевой график проекта

Таким образом, построение диаграммы Ганта, графика загрузки ресурсов и сетевого графика позволяет получить целостное представление о ходе реализации проекта, определить временные и ресурсные ограничения, а также выделить этапы, оказывающие наибольшее влияние на сроки разработки продукта.

7 Анализ затрат на реализацию проекта

7.1 Анализ затрат на изготовление продукта

После определения трудоёмкости работ, состава исполнителей и календарного графика выполнения проекта необходимо выполнить расчёт затрат на разработку программного продукта. Структура затрат должна отражать основные ресурсы, необходимые для создания разрабатываемой системы, и включать расходы на оплату труда исполнителей, оборудование, амортизационные отчисления, расходные материалы, организацию рабочих мест и накладные расходы.

Общая величина затрат на изготовление продукта определяется по формуле:

$$C = C_{\text{з.п.}} + C_{\text{об}} + C_{\text{аморт}} + C_{\text{расх}} + C_{\text{орг}} + C_{\text{накл}}. \tag{21}$$

Затраты на заработную плату исполнителей

Затраты на заработную плату определяются как сумма основной заработной платы, дополнительной заработной платы и страховых отчислений:

$$C_{\text{з.п.}} = C_{\text{осн}} + C_{\text{доп}} + C_{\text{отч}}. \tag{22}$$

Дополнительная заработная плата составляет 20% от основной:

$$C_{\text{доп}} = 0.2 \cdot C_{\text{осн}}. \tag{23}$$

Страховые отчисления составляют 30% от суммы основной и дополнительной заработной платы:

$$C_{\text{отч}} = (C_{\text{осн}} + C_{\text{доп}}) \cdot 0.3. \tag{24}$$

Расчёт основной заработной платы выполняется на основе трудоёмкости работ и месячных окладов исполнителей. Часовая тарифная ставка определяется исходя из месячного фонда рабочего времени 168 часов:

$$C_{\text{час}} = \frac{C_{\text{мес}}}{168}. \tag{25}$$

При расчёте фонда заработной платы необходимо учитывать не только суммарную трудоёмкость проекта, равную 436 чел.-ч, но и фактическое распределение работ между исполнителями. В рассматриваемом проекте этапы анализа требований и подготовки документации выполняются аналитиком, проектирование архитектуры и проработка взаимодействия модулей — архитектором, реализация библиотеки трассировки — старшим разработчиком, а разработка подсистем агрегации и обработки трасс — разработчиком. При этом этап тестирования и отладки выполняется совместно разработчиком и старшим разработчиком, поэтому его трудоёмкость учитывается в фонде рабочего времени обоих исполнителей. В результате суммарные трудозатраты по исполнителям составляют 480 чел.-ч, что превышает общую трудоёмкость проекта за счёт параллельного участия двух специалистов в заключительном этапе.

Для расчёта примем следующие значения месячных окладов: аналитик — 120000 руб., архитектор — 150000 руб., разработчик — 140000 руб., старший разработчик — 170000 руб.

Таблица 7 – Расчёт основной заработной платы исполнителей

№	Должность	Оклад, руб.	Часовая ставка, руб./ч	Трудозатраты, ч	Трудозатраты, дни	Оплата труда, руб.
1	Аналитик	120000	714.29	116	14.5	82857.14
2	Архитектор	150000	892.86	84	10.5	75000.00
3	Разработчик	140000	833.33	124	15.5	103333.33
4	Старший разработчик	170000	1011.90	156	19.5	157857.14
					Итого основная заработная плата $C_{\text{осн}}$	419047.61

Тогда дополнительная заработная плата составит:

$$C_{\text{доп}} = 0.2 \cdot 419047.61 = 83809.52 \text{ руб.} \tag{26}$$

Страховые отчисления:

$$C_{\text{отч}} = (419047.61 + 83809.52) \cdot 0.3 = 150857.14 \text{ руб.} \tag{27}$$

Полные затраты на заработную плату исполнителей:

$$C_{\text{з.п.}} = 419047.61 + 83809.52 + 150857.14 = 653714.27 \text{ руб.} \tag{28}$$

Затраты на оборудование

Для реализации проекта требуется использование вычислительной техники, необходимой для разработки, тестирования и выполнения программного инструмента. В отличие от проектов, ориентированных на эксплуатацию в крупной инфраструктуре, разрабатываемая система не требует выделенных серверных мощностей и может быть реализована на персональных компьютерах исполнителей.

Тем не менее, для обеспечения полноценного процесса разработки и тестирования необходимо предусмотреть рабочие станции для участников проекта, а также вспомогательное оборудование для хранения данных и резервного копирования.

Состав необходимого оборудования и затраты на его приобретение приведены в таблице 8.

Таблица 8 – Затраты на закупку оборудования и технических средств

№	Наименование оборудования	Количество	Цена за единицу, руб.	Сумма, руб.
1	Ноутбук для разработки	4	80000	320000
2	Рабочая станция старшего разработчика (повышенной производительности)	1	120000	120000
3	Внешний накопитель для хранения и резервного копирования данных	1	10000	10000
4	Источник бесперебойного питания	1	15000	15000
5	Сетевое оборудование (маршрутизатор)	1	8000	8000
			Итого $C_{об}$:	473000

Таким образом, затраты на закупку оборудования и технических средств составляют:

$$C_{об} = 473000 \text{ руб.} \tag{29}$$

Амортизационные расходы на оборудование

Поскольку оборудование используется в проекте в течение ограниченного периода времени, необходимо определить амортизационные расходы за время выполнения работ.

Амортизационные расходы рассчитываются по формуле:

$$C_{\text{аморт}} = \sum_i \frac{C_{\text{об}i} \cdot \Delta t_i}{D_{\text{раб}i}}, \tag{30}$$

где $C_{\text{об}i}$ — стоимость i-го оборудования; Δt_i — время использования оборудования в днях; $D_{\text{раб}i}$ — срок эксплуатации оборудования в рабочих днях.

Примем, что срок службы вычислительной техники составляет 5 лет, а вспомогательного оборудования — 10 лет. При количестве рабочих дней в году, равном 250, получаем:

$$D_{\text{раб}}^{(1)} = 5 \cdot 250 = 1250 \text{ дней}, \quad D_{\text{раб}}^{(2)} = 10 \cdot 250 = 2500 \text{ дней}.$$

Продолжительность проекта согласно календарному плану составляет 43 рабочих дня.

Расчёт амортизационных расходов приведён в таблице 9.

Таблица 9 – Расчёт амортизационных расходов по оборудованию

№	Наименование оборудования	Стоимость, руб.	Срок эксплуатации, дни	Амортизация за проект, руб.
1	Ноутбук для разработки (4 шт.)	320000	1250	11008.00
2	Рабочая станция старшего разработчика	120000	1250	4128.00
3	Внешний накопитель	10000	2500	172.00
4	Источник бесперебойного питания	15000	2500	258.00
5	Маршрутизатор	8000	2500	137.60
			Итого $C_{\text{аморт}}$:	15703.60

Следовательно, амортизационные расходы на оборудование составляют:

$$C_{\text{аморт}} = 15703.60 \text{ руб.} \tag{31}$$

Затраты на расходные материалы

Затраты на расходные материалы определяются перечнем минимально необходимых материалов, используемых в ходе выполнения проекта. Поскольку разрабатываемый продукт является программным средством, основная часть расходных материалов связана с подготовкой документации, организацией рабочего процесса и переносом данных.

Расчёт затрат на расходные материалы приведён в таблице 10.

Таблица 10 – Затраты на расходные материалы

№	Наименование материалов	Цена, руб.	Количество	Сумма, руб.
1	Бумага А4, 500 листов	300	3	900
2	Набор маркеров	350	2	700
3	Канцелярские принадлежности	500	2	1000
4	USB-накопитель 32 ГБ	600	3	1800
			Итого $C_{\text{расх}}$:	4400

Следовательно, затраты на расходные материалы составляют:

$$C_{\text{расх}} = 4400 \text{ руб.} \tag{32}$$

Затраты на организацию рабочих мест

Затраты на организацию рабочих мест определяются на основе требований к площади рабочих мест и стоимости аренды офисного помещения. Для размещения одного рабочего места с персональным компьютером необходимо не менее 6 кв. м площади.

В проекте задействованы 4 исполнителя (аналитик, архитектор, разработчик и старший разработчик), следовательно минимальная необходимая площадь помещения составляет:

$$S = 4 \cdot 6 = 24 \text{ кв. м.}$$

Рассмотрим несколько вариантов аренды офисного помещения.

Таблица 11 – Варианты аренды офисного помещения

№	Район	Площадь, кв. м	Стоимость, руб./кв. м/год
1	м. Бауманская	25	14000
2	м. Электрозаводская	30	9000
3	м. Семёновская	28	11000

Для дальнейших расчётов выбирается помещение у станции метро «Электрозаводская», так как оно удовлетворяет требованиям по площади и имеет минимальную стоимость аренды.

Затраты на аренду помещения рассчитываются по формуле:

$$C_{\text{орг}} = \frac{C_{\text{кв.м.}} \cdot S \cdot T_{\text{ар}}}{12}, \tag{33}$$

где $C_{\text{кв.м.}}$ — стоимость аренды одного квадратного метра в год; S — площадь помещения; $T_{\text{ар}}$ — срок аренды в месяцах.

Продолжительность проекта составляет примерно 2 месяца, тогда:

$$C_{\text{орг}} = \frac{9000 \cdot 30 \cdot 2}{12} = 45000 \text{ руб.}$$

Таким образом, затраты на организацию рабочих мест составляют:

$$C_{\text{орг}} = 45000 \text{ руб.} \tag{34}$$

Накладные расходы

Накладные расходы, связанные с выполнением проекта, определяются на основе основной заработной платы исполнителей. В соответствии с методикой их величина принимается равной 60% от основной заработной платы.

Следовательно:

$$C_{\text{накл}} = 0.6 \cdot C_{\text{осн}}. \tag{35}$$

Подставляя рассчитанное ранее значение основной заработной платы $C_{\text{осн}} = 419047.61$ руб., получаем:

$$C_{\text{накл}} = 0.6 \cdot 419047.61 = 251428.57 \text{ руб.}$$

Таким образом, накладные расходы составляют:

$$C_{\text{накл}} = 251428.57 \text{ руб.} \tag{36}$$

7.2 Анализ затрат на внедрение продукта

Затраты на внедрение результатов проекта включают расходы, связанные с практическим применением разработанной методики в реальных программных системах. В отличие от этапа разработки, внедрение носит длительный и итерационный характер и предполагает использование методики в процессе сопровождения программного обеспечения.

В рамках внедрения выполняются следующие работы:

– разработка TLA+ спецификаций для отдельных компонентов системы;
– инструментирование исходного кода для получения трасс исполнения;
– проверка соответствия реализации формальной спецификации;
– тестирование и уточнение TLA+ спецификаций;
– анализ результатов проверки и устранение выявленных несоответствий.

Затраты на внедрение определяются по формуле:

$$K_{\text{вн}} = C_{\text{вн.зп}} + C_{\text{вн.обор}} + C_{\text{вн.орг}} + C_{\text{вн.накл}}. \tag{37}$$

Дополнительное оборудование для внедрения не требуется, так как используется ранее приобретённая вычислительная техника:

$$C_{\text{вн.обор}} = 0.$$

Внедрение выполняется в течение двух лет. Работы выполняются двумя исполнителями: разработчиком и старшим разработчиком. Предполагается, что они задействованы в среднем на 25% рабочего времени.

Годовой фонд рабочего времени принимается равным 2000 часов. Тогда суммарная трудоёмкость внедрения составляет:

$$T_{\text{вн}} = 2 \cdot 2000 \cdot 0.25 = 1000 \text{ чел.-ч.}$$

Распределим трудоёмкость между исполнителями поровну:

$$T_{\text{разр}} = 500 \text{ ч}, \quad T_{\text{ст.разр}} = 500 \text{ ч}.$$

Тогда основная заработная плата составит:

$$C_{\text{осн}}^{\text{вн}} = 500 \cdot 833.33 + 500 \cdot 1011.90.$$

После вычисления получаем:

$$C_{\text{осн}}^{\text{вн}} = 416665 + 505950 = 922615 \text{ руб.}$$

Дополнительная заработная плата:

$$C_{\text{доп}}^{\text{вн}} = 0.2 \cdot 922615 = 184523 \text{ руб.}$$

Страховые отчисления:

$$C_{\text{отч}}^{\text{вн}} = (922615 + 184523) \cdot 0.3 = 332141.4 \text{ руб.}$$

Таким образом, затраты на заработную плату при внедрении составляют:

$$C_{\text{вн.зп}} = 922615 + 184523 + 332141.4 = 1439279.4 \text{ руб.} \tag{38}$$

Затраты на организацию рабочих мест определяются исходя из количества исполнителей. Для двух рабочих мест минимальная площадь составляет:

$$S = 2 \cdot 6 = 12 \text{ кв. м.}$$

Примем помещение площадью 15 кв. м со стоимостью аренды 9000 руб./кв. м/год. Срок аренды — 24 месяца. Тогда:

$$C_{\text{вн.орг}} = \frac{9000 \cdot 15 \cdot 24}{12} = 270000 \text{ руб.}$$

Накладные расходы принимаются равными 90% от основной заработной платы:

$$C_{\text{вн.накл}} = 0.9 \cdot 922615 = 830353.5 \text{ руб.} \tag{39}$$

Тогда общие затраты на внедрение составляют:

$$K_{\text{вн}} = 1439279.4 + 0 + 270000 + 830353.5 = 2539632.9 \text{ руб.} \tag{40}$$

7.3 Общие затраты

Общие затраты на реализацию проекта включают затраты на изготовление программного продукта и затраты на его внедрение. Совокупная стоимость проекта определяется по формуле:

$$K_{\text{общ}} = C + K_{\text{вн}}. \tag{41}$$

Сведём затраты на изготовление продукта в таблицу.

Таким образом, затраты на изготовление программного продукта составляют:

Таблица 12 – Структура затрат на изготовление продукта

Статья затрат	Сумма, руб.
Заработная плата исполнителей $C_{\text{з.п.}}$	653714.27
Оборудование $C_{\text{об}}$	473000
Амортизация $C_{\text{аморт}}$	15703.60
Расходные материалы $C_{\text{расх}}$	4400
Организация рабочих мест $C_{\text{орг}}$	45000
Накладные расходы $C_{\text{накл}}$	251428.57
Итого C	1443246.44

$$C = 1443246.44 \text{ руб.} \tag{42}$$

Структура затрат на внедрение проекта приведена в таблице 13.

Таблица 13 – Структура затрат на внедрение продукта

Статья затрат	Сумма, руб.
Заработная плата исполнителей $C_{\text{вн.зп}}$	1439279.40
Оборудование $C_{\text{вн.обор}}$	0
Организация рабочих мест $C_{\text{вн.орг}}$	270000
Накладные расходы $C_{\text{вн.накл}}$	830353.50
Итого $K_{\text{вн}}$	2539632.90

Следовательно, общие затраты на реализацию проекта составляют:

$$K_{\text{общ}} = 1443246.44 + 2539632.90 = 3982879.34 \text{ руб.} \tag{43}$$

Для наглядного представления структуры затрат на изготовление и внедрение проекта построены круговые диаграммы (рис. 7 и рис. 8).

Таким образом, наибольшую долю в структуре затрат занимают расходы на заработную плату исполнителей и накладные расходы, что обусловлено интеллектуальной природой разрабатываемого программного продукта. Внедрение требует существенно больших затрат по сравнению с этапом разработки, что связано с длительным сроком применения методики и необходимостью участия квалифицированных специалистов.

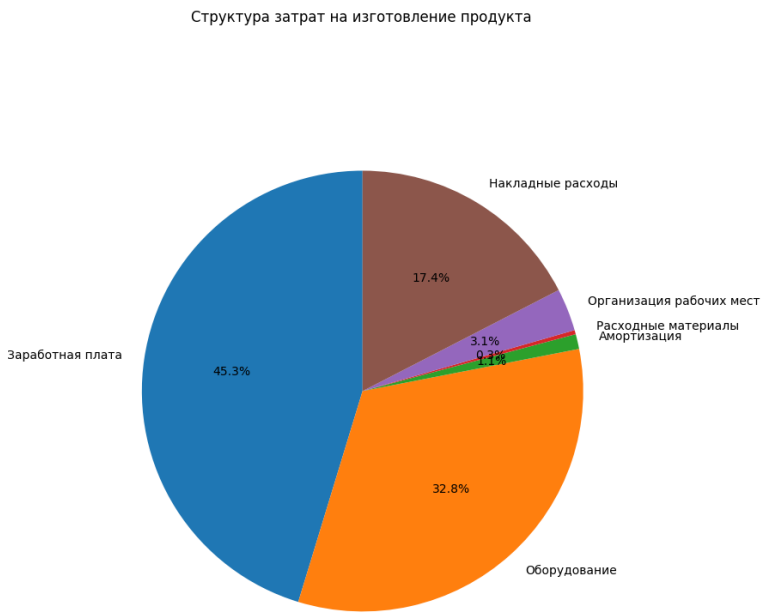

Рисунок 7 – Структура затрат на изготовление продукта

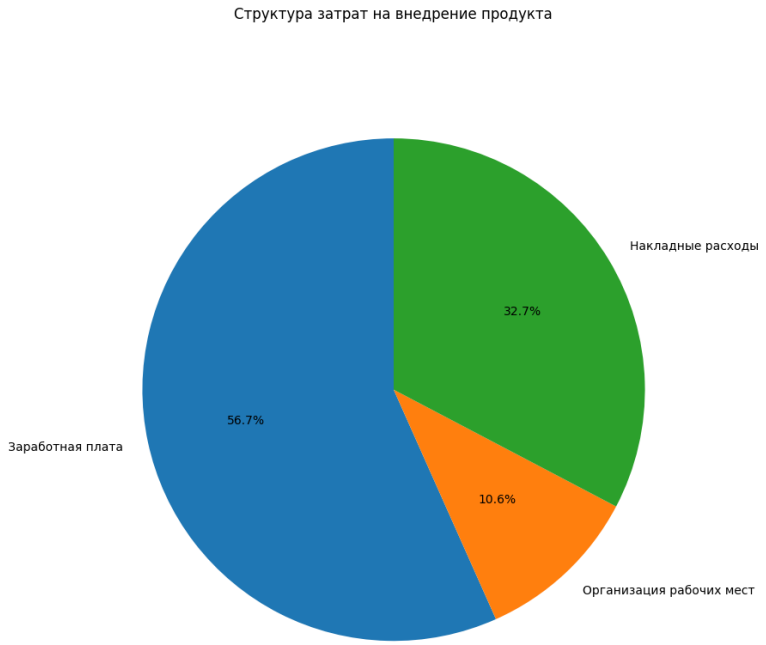

Рисунок 8 – Структура затрат на внедрение продукта

8 Исследование рынка

Исследование сегмента рынка, на который ориентирован разрабатываемый продукт, направлено на определение области его применения, оценку потенциального спроса и прогнозирование количества возможных продаж.

Разрабатываемый продукт представляет собой специализированный инструмент и методику верификации соответствия реализации формальным спецификациям на языке TLA+. Основной областью применения являются организации, разрабатывающие сложные программные системы с высокими требованиями к корректности: распределённые системы, финансовые сервисы, системы хранения данных и инфраструктурные решения.

В настоящее время применение формальных методов верификации остаётся ограниченным из-за высокой сложности внедрения, необходимости квалифицированных специалистов и отсутствия удобных инструментов интеграции с существующим кодом. В частности, использование TLA+ требует:

- разработки формальной спецификации системы;
- сопоставления абстрактной модели с реальной реализацией;
- инструментирования кода для получения трасс исполнения;
- анализа результатов модельной проверки.

Разрабатываемый продукт решает задачу сопоставления трасс исполнения с формальной спецификацией, однако его внедрение остаётся сложным процессом, требующим участия квалифицированных разработчиков и аналитиков. Это существенно ограничивает массовость применения и снижает число потенциальных покупателей, одновременно увеличивая ценность продукта для каждой конкретной организации.

По оценкам, ежегодно реализуется порядка 3000 проектов, из которых около 5% могут потенциально использовать формальные методы верификации:

$$N_p = 0.05 \cdot 3000 = 150 \text{ проектов в год.} \tag{44}$$

С учётом высокой сложности внедрения, специфичности продукта и необходимости адаптации под конкретные системы, примем долю реальных покупок на уровне 10% от потенциального спроса:

$$N_p^o = 0.1 \cdot 150 = 15 \text{ продаж в год.} \tag{45}$$

Средняя стоимость одной копии продукта определяется его специализированным характером и включает не только стоимость программного обеспечения, но и затраты на внедрение, интеграцию и сопровождение. В отличие от массовых программных продуктов, подобные решения поставляются как комплексный инструмент, ориентированный на решение критически важных задач.

Высокая стоимость обусловлена следующими факторами:

- необходимость адаптации инструмента под конкретную систему;
- сложность разработки и сопровождения TLA+ спецификаций;
- высокая квалификация специалистов, участвующих во внедрении;
- значительная ценность продукта для предотвращения ошибок в критических системах.

С учётом указанных факторов примем среднюю стоимость одной копии:

$$K_{\text{пр}} = 250000 \text{ руб.} \tag{46}$$

Тогда прогнозируемый объём продаж в год составит:

$$V = N_p^o \cdot K_{\text{пр}} = 15 \cdot 250000 = 3750000 \text{ руб.} \tag{47}$$

За период в два года суммарный объём продаж составит:

$$V_2 = 2 \cdot 3750000 = 750000 \text{ руб.} \tag{48}$$

Таким образом, несмотря на сравнительно небольшое количество продаж, высокая стоимость продукта обеспечивает значительный объём выручки. Это объясняется тем, что продукт ориентирован на узкий сегмент рынка и представляет собой высокоспециализированное решение с высокой добавленной стоимостью.

9 Планирование цены и прогнозирование прибыли

На основе рассчитанных затрат на изготовление и внедрение программного продукта, а также результатов исследования рынка, необходимо определить экономически обоснованную цену одной копии продукта и оценить ожидаемую прибыль от его реализации.

Стоимость программного продукта определяется с учётом части затрат на разработку, приходящейся на одну копию, затрат на внедрение и заложенной прибыли. Расчёт выполняется по формуле:

$$K_{\text{по}} = (\Delta K + K_{\text{вн}}) \cdot (1 + D_{\text{приб}}), \tag{49}$$

где ΔK — часть стоимости разработки, приходящаяся на одну копию продукта; $K_{\text{вн}}$ — стоимость внедрения одной копии; $D_{\text{приб}}$ — доля прибыли, заложенная в цену.

Частичная стоимость разработки одной копии определяется по формуле:

$$\Delta K = \frac{K}{N_p^o} \cdot (1 + H_{\text{ст}}), \tag{50}$$

где K — затраты на изготовление продукта; N_p^o — планируемое число продаж за расчётный период; $H_{\text{ст}}$ — ставка банковского процента по долгосрочному кредиту.

Так как оценка прибыльности проводится на горизонте двух лет, при планируемом объёме продаж 15 копий в год примем:

$$N_p^o = 30 \text{ копий.}$$

Из предыдущих расчётов:

$$K = 1443246.44 \text{ руб.}, \qquad K_{\text{вн,общ}} = 2539632.90 \text{ руб.}, \qquad H_{\text{ст}} = 0.25.$$

Тогда часть затрат на изготовление, приходящаяся на одну копию, составит:

$$\Delta K = \frac{1443246.44}{30} \cdot 1.25 \approx 60135.27 \text{ руб.}$$

Стоимость внедрения одной копии определяется как:

$$K_{\text{вн}} = \frac{2539632.90}{30} \approx 84654.43 \text{ руб.}$$

Следовательно, полная себестоимость одной копии продукта составляет:

$$K_{\text{себ}} = \Delta K + K_{\text{вн}} = 60135.27 + 84654.43 = 144789.70 \text{ руб.} \tag{51}$$

С учётом принятой рыночной цены одной копии:

$$K_{\text{пр}} = 250000 \text{ руб.},$$

можно определить долю прибыли:

$$D_{\text{приб}} = \left(\frac{K_{\text{пр}}}{K_{\text{себ}}} - 1 \right). \tag{52}$$

Подставляя значения, получаем:

$$D_{\text{приб}} = \left(\frac{250000}{144789.70} - 1 \right) \approx 0.726,$$

или в процентном выражении:

$$D_{\text{приб}} \approx 72.6\%.$$

Полученное значение является более реалистичным для узкоспециализированного программного продукта, ориентированного на корпоративный рынок. Такая норма прибыли учитывает как необходимость окупаемости затрат на разработку и внедрение, так и ограниченный объём рынка.

Прибыль от реализации одной копии определяется по формуле:

$$C_{\text{приб}} = K_{\text{пр}} \cdot D_{\text{приб}} \cdot (1 - H_{\text{ндс}}), \tag{53}$$

где $H_{\text{ндс}} = 0.2$ — ставка налога на добавленную стоимость.

Тогда прибыль от реализации одной копии составит:

$$C_{\text{приб}} = 250000 \cdot 0.726 \cdot 0.8 \approx 145200 \text{ руб.}$$

Годовая выручка при реализации 15 копий составит:

$$V = 15 \cdot 250000 = 3750000 \text{ руб.}$$

За два года выручка составит:

$$V_2 = 30 \cdot 250000 = 7500000 \text{ руб.}$$

Годовая прибыль при реализации 15 копий составит:

$$C_{\text{год}} = 15 \cdot 145200 = 2178000 \text{ руб.}$$

За два года:

$$C_2 = 30 \cdot 145200 = 4356000 \text{ руб.}$$

Для оценки изменения денежных потоков во времени составим фрагмент общего баланса проекта с учётом выполненных расчётов. Расчётный период разобьём на кварталы. Для более реалистичного отражения динамики продаж учтём, что в первые месяцы проект находится на стадии завершения разработки и подготовки к внедрению. В связи с этим в первые 3 месяца осуществляется только одна продажа, после чего продажи стабилизируются. Предположим, что далее продажи распределяются равномерно, и за оставшиеся периоды реализуется оставшийся объём продукта.

Расчёт показателей таблицы 14 выполняется последовательно для каждого расчётного периода.

Для i-го периода используются следующие зависимости:

– сумма продаж:

$$V_i = n_i \cdot K_{\text{пр}}, \tag{54}$$

где n_i — количество проданных копий в периоде;

– затраты на внедрение:

$$C_{\text{вн},i} = n_i \cdot K_{\text{вн}}, \tag{55}$$

– погашение затрат на разработку:

$$C_{\text{разр},i} = n_i \cdot \Delta K, \tag{56}$$

– чистая прибыль:

$$P_i = n_i \cdot C_{\text{приб}}, \tag{57}$$

– конечный баланс:

$$B_i = B_{i-1} + P_i, \tag{58}$$

где B_{i-1} — баланс на начало периода.

Рассмотрим пример расчёта для периода «4–6 месяцев», в котором реализуется $n_2 = 3$ копии продукта.

Сумма продаж:

$$V_2 = 3 \cdot 250000 = 750000 \text{ руб.}$$

Затраты на внедрение:

$$C_{\text{вн},2} = 3 \cdot 84654.43 = 253963.29 \text{ руб.}$$

Погашение затрат на разработку:

$$C_{\text{разр},2} = 3 \cdot 60135.27 = 180405.81 \text{ руб.}$$

Чистая прибыль:

$$P_2 = 3 \cdot 105210.30 = 315630.89 \text{ руб.}$$

Баланс на начало периода:

$$B_1 = -1338036.14 \text{ руб.}$$

Тогда конечный баланс:

$$B_2 = -1338036.14 + 315630.89 = -1022405.25 \text{ руб.}$$

Аналогичным образом выполняются расчёты для остальных периодов.

Таблица 14 – Фрагмент общего баланса проекта с учётом выполненных расчётов

Период расчёта	Баланс начальный, руб.	Сумма продаж, руб.	Сумма погашения кредита на внедрение, руб.	Погашение кредита на разработку, руб.	Чистая прибыль, руб.	Баланс конечный, руб.
1–3 мес.	−1443246.44	250000.00	84654.43	60135.27	105210.30	−1338036.14
4–6 мес.	−1338036.14	750000.00	253963.29	180405.81	315630.89	−1022405.25
7–9 мес.	−1022405.25	1000000.00	338617.72	240541.08	420841.19	−601564.06
10–12 мес.	−601564.06	1000000.00	338617.72	240541.08	420841.19	−180722.87
13–15 мес.	−180722.87	1000000.00	338617.72	240541.08	420841.19	240118.32
16–18 мес.	240118.32	1000000.00	338617.72	240541.08	420841.19	660959.51
19–21 мес.	660959.51	1250000.00	423272.15	300676.35	526051.48	1187010.99
22–24 мес.	1187010.99	1250000.00	423272.15	300676.35	526051.49	1713062.48

Как видно из таблицы 14, в первые три месяца проект остаётся убыточным, что связано с завершением разработки и началом внедрения, вследствие чего в данный период реализуется только одна копия продукта. В последующие периоды объём продаж возрастает, а проект достигает точки безубыточности к концу первого года эксплуатации.

Дальнейшее увеличение количества продаж обеспечивает устойчивый рост прибыли, что подтверждает экономическую целесообразность реализации проекта при выбранной цене продукта.

Для наглядного представления динамики прибыли может быть построен график изменения баланса и чистой прибыли по расчётным периодам.

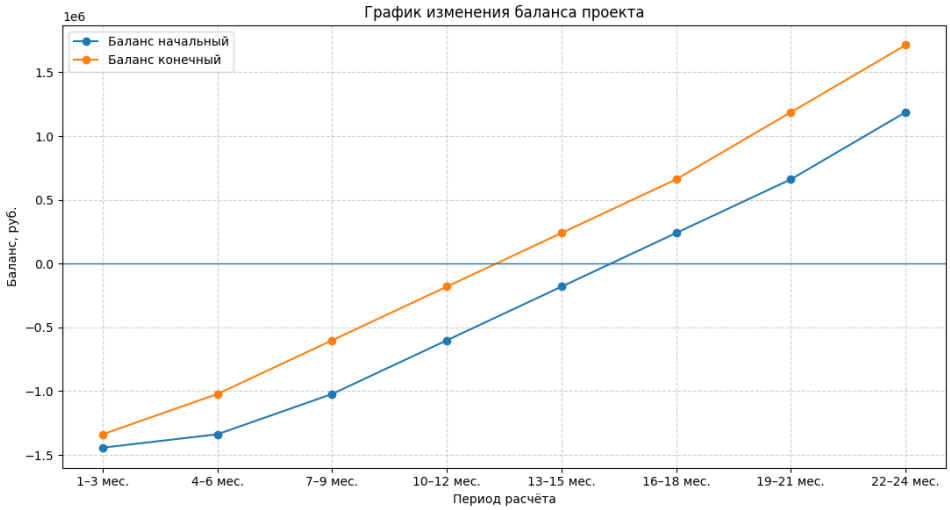

Рисунок 9 – График изменения баланса проекта

Таким образом, выбранная цена продукта в размере 250000 руб. обеспечивает покрытие затрат на изготовление и внедрение проекта и позволяет получить положительный финансовый результат на горизонте двух лет. При этом уровень прибыли остаётся умеренным и выглядит более реалистично для специализированного программного продукта, ориентированного на ограниченный сегмент рынка.

10 Цифровая оценка инвестиций в инновационное производство

Цифровая оценка инвестиций в инновационное производство направлена на определение экономической эффективности разработки программного продукта и обоснование целесообразности вложения средств в его создание. Для этого используются показатели, основанные на анализе денежных потоков проекта и учёте фактора времени.

В рамках данного раздела для оценки инвестиционной привлекательности проекта используются следующие показатели:

- чистая приведённая стоимость (NPV);
- внутренняя норма доходности (IRR);
- рентабельность инвестиций (ROI);
- экономическая добавленная стоимость (EVA);
- срок окупаемости проекта.

В качестве исходного объёма инвестиций принимаются затраты на изготовление продукта:

$$K = 1443246.44 \text{ руб.} \tag{59}$$

Согласно результатам предыдущего раздела, годовая прибыль проекта составляет:

$$C_{\text{год}} = 2178000 \text{ руб.} \tag{60}$$

Для расчётов примем, что прибыль поступает равномерно в течение двух лет, а ставка дисконтирования составляет:

$$r = 0.25. \tag{61}$$

Чистая приведённая стоимость

Чистая приведённая стоимость определяется по формуле:

$$NPV = -K + \sum_{t=1}^{n} \frac{CF_t}{(1+r)^t}, \tag{62}$$

где CF_t — денежный поток в период t; n — число расчётных периодов.

Для рассматриваемого проекта принимаем:

$$CF_1 = CF_2 = 2178000 \text{ руб.}, \qquad n = 2.$$

Тогда:

$$NPV = -1443246.44 + \frac{2178000}{1.25} + \frac{2178000}{1.25^2}. \tag{63}$$

После вычисления получаем:

$$NPV = 1693073.56 \text{ руб.} \tag{64}$$

Положительное значение NPV показывает, что проект обладает инвестиционной привлекательностью и обеспечивает прирост стоимости вложенного капитала.

Внутренняя норма доходности

Внутренняя норма доходности определяется как такое значение ставки дисконтирования, при котором чистая приведённая стоимость равна нулю:

$$0 = -K + \sum_{t=1}^{n} \frac{CF_t}{(1 + IRR)^t}. \tag{65}$$

Для рассматриваемого проекта численным методом получаем:

$$IRR \approx 119.62\%. \tag{66}$$

Значение IRR существенно превышает принятую ставку дисконтирования, что свидетельствует о высокой эффективности инвестиций в разработку продукта.

Рентабельность инвестиций

Рентабельность инвестиций определяется как отношение прибыли к объёму инвестиций:

$$ROI = \frac{C_2 - K}{K} \cdot 100\%, \tag{67}$$

где C_2 — прибыль за два года реализации проекта.

Так как:

$$C_2 = 4356000 \text{ руб.},$$

получаем:

$$ROI = \frac{4356000 - 1443246.44}{1443246.44} \cdot 100\% \approx 201.82\%. \tag{68}$$

Следовательно, за два года проект приносит прибыль, более чем в два раза превышающую первоначальные инвестиции.

Экономическая добавленная стоимость

Экономическая добавленная стоимость рассчитывается по формуле:

$$EVA = P - WACC \cdot K, \tag{69}$$

где P — чистая прибыль за период; $WACC$ — средневзвешенная стоимость капитала.

Примем:

$$WACC = 0.2.$$

Тогда при годовой прибыли $P = 2178000$ руб. получаем:

$$EVA = 2178000 - 0.2 \cdot 1443246.44 = 1889350.71 \text{ руб.} \tag{70}$$

Положительное значение EVA показывает, что проект создаёт добавленную стоимость и экономически оправдан с позиции инвестора.

Срок окупаемости

Срок окупаемости определяется как отношение объёма инвестиций к среднегодовой прибыли:

$$T_{\text{ок}} = \frac{K}{C_{\text{год}}}. \tag{71}$$

Тогда:

$$T_{\text{ок}} = \frac{1443246.44}{2178000} \approx 0.66 \text{ года}. \tag{72}$$

В пересчёте на месяцы срок окупаемости составляет:

$$T_{\text{ок}} \approx 7.95 \text{ месяцев.} \tag{73}$$

Это означает, что вложения в разработку продукта возвращаются менее чем за один год эксплуатации.

Учёт затрат по видам деятельности

Основные методы учёта затрат включают:

- учёт затрат по видам деятельности;
- полную стоимость владения.

Учёт затрат по видам деятельности представляет собой метод управленческого учёта, позволяющий оценить затраты на различные виды деятельности в рамках инновационного процесса. Данный подход даёт возможность выделить прямые и косвенные расходы, возникающие на этапах разработки, внедрения и последующей эксплуатации продукта.

Для рассматриваемого проекта в качестве прямых затрат выступают затраты на заработную плату исполнителей, оборудование, амортизационные отчисления, расходные материалы и организацию рабочих мест. Косвенные затраты представлены накладными расходами, связанными с управлением проектом, сопровождением и обеспечением взаимодействия между участниками процесса.

Для укрупнённой оценки затрат по видам деятельности используем показатель полной стоимости владения (*Total Cost of Ownership*, TCO), который определяется как сумма затрат на изготовление продукта и затрат на внедрение всех планируемых экземпляров:

$$TCO = K + N_p^o \cdot K_{\text{вн}}, \tag{74}$$

где K — стоимость изготовления продукта; N_p^o — планируемое количество реализуемых экземпляров; $K_{\text{вн}}$ — стоимость внедрения одного экземпляра.

Согласно предыдущим расчётам:

$$K = 1443246.44 \text{ руб.}, \qquad N_p^o = 30, \qquad K_{\text{вн}} = 84654.43 \text{ руб.}$$

Тогда:

$$TCO = 1443246.44 + 30 \cdot 84654.43 = 3982879.34 \text{ руб.} \tag{75}$$

Полученное значение показывает полную совокупную стоимость владения продуктом на горизонте двух лет, включая затраты на его изготовление и последующее внедрение в планируемом объёме.

Оценка использования инновации на уровне предприятия

Оценка использования инновации на уровне предприятия позволяет определить, насколько внедрение разрабатываемого решения способствует повышению эффективности использования ресурсов организации.

Одним из показателей такой оценки является показатель производительности инновации IP, который определяется как отношение экономической добавленной стоимости к коммерческим, общим и административным расходам:

$$IP = \frac{EVA}{SG\&A}, \tag{76}$$

где EVA — экономическая добавленная стоимость; $SG\&A$ — коммерческие, общие и административные расходы.

В рамках укрупнённой оценки в качестве $SG\&A$ примем суммарные накладные расходы по проекту, включающие накладные расходы на этапе изготовления и на этапе внедрения:

$$SG\&A = C_{\text{накл}} + C_{\text{вн.накл}} = 251428.57 + 830353.50 = 1081782.07 \text{ руб.}$$

Экономическая добавленная стоимость ранее была рассчитана как:

$$EVA = 1889350.71 \text{ руб.}$$

Тогда:

$$IP = \frac{1889350.71}{1081782.07} \approx 1.75 \tag{77}$$

Поскольку полученное значение превышает единицу, можно сделать вывод, что внедрение разработанного решения положительно влияет на эффективность использования ресурсов предприятия. Иными словами, создаваемая инновация обеспечивает экономический эффект, превышающий уровень сопутствующих административных и накладных расходов.

Итоговая оценка инвестиционной привлекательности проекта

Для удобства сведём полученные показатели в таблицу.

Таблица 15 – Показатели эффективности инвестиций проекта

Показатель	Значение
Чистая приведённая стоимость NPV	1693073.56 руб.
Внутренняя норма доходности IRR	119.62%
Рентабельность инвестиций ROI	201.82%
Экономическая добавленная стоимость EVA	1889350.71 руб.
Полная стоимость владения TCO	3982879.34 руб.
Показатель использования инновации IP	1.75
Срок окупаемости	7.95 месяцев

Таким образом, проведённая цифровая оценка инвестиций показывает, что проект является экономически эффективным. Положительное значение NPV, высокий уровень IRR, положительная величина EVA, значение показателя IP, превышающее единицу, и короткий срок окупаемости подтверждают целесообразность инвестирования в разработку рассматриваемого программного продукта.

11 Заключение

В организационно-экономической части работы была выполнена оценка трудоёмкости, стоимости разработки, затрат на внедрение, рыночных параметров продукта и показателей инвестиционной эффективности разрабатываемой методики верификации соответствия TLA+ спецификации и исходного кода.

В результате проведённых расчётов установлено, что суммарная трудоёмкость разработки составляет 436 чел.-ч, что соответствует 54.5 чел.-дням. Календарная продолжительность проекта по диаграмме Ганта составляет 43 рабочих дня. Наиболее трудоёмким этапом является разработка библиотеки инструментирования исходного кода, трудоёмкость которой составляет 112 чел.-ч.

Расчёт затрат показал, что стоимость изготовления программного продукта составляет 1443246.44 руб., а затраты на внедрение продукта — 2539632.90 руб. Следовательно, общие затраты на реализацию проекта составляют 3982879.34 руб. Наибольшую долю в структуре затрат занимают расходы на заработную плату исполнителей, оборудование и накладные расходы.

В ходе исследования рынка установлено, что продукт ориентирован на узкий сегмент корпоративных заказчиков, использующих формальные методы верификации. Планируемый объём продаж составляет 15 копий в год, или 30 копий за двухлетний период. Принятая рыночная цена одной копии продукта составляет 250000 руб. При этом годовая выручка прогнозируется на уровне 3750000 руб., а выручка за два года — 7500000 руб.

Расчёты показали, что полная себестоимость одной копии продукта составляет 144789.70 руб., а прибыль от реализации одной копии — 145200 руб. Годовая прибыль оценивается в 2178000 руб., а прибыль за два года — в 4356000 руб. Согласно балансу проекта, точка безубыточности достигается примерно на 13–15 месяце эксплуатации.

Дополнительно была выполнена цифровая оценка инвестиций в инновационное производство. При объёме инвестиций в изготовление продукта $K = 1443246.44$ руб. получены следующие показатели: чистая приведённая стоимость $NPV = 1693073.56$ руб., внутренняя норма доходности $IRR = 119.62\%$, рентабельность инвестиций $ROI = 201.82\%$, экономическая добавленная стоимость $EVA = 1889350.71$ руб., срок окупаемости — 7.95 месяцев.

Основные результаты организационно-экономического обоснования приведены в таблице 16.

Таблица 16 – Итоговые технико-экономические показатели проекта

Показатель	Значение
Суммарная трудоёмкость разработки	436 чел.-ч
Трудоёмкость разработки в человеко-днях	54.5 чел.-дня
Календарная продолжительность проекта	43 рабочих дня
Наиболее трудоёмкий этап	Разработка библиотеки трассировки, 112 чел.-ч
Затраты на изготовление продукта	1443246.44 руб.
Затраты на внедрение продукта	2539632.90 руб.
Общие затраты на реализацию проекта	3982879.34 руб.
Цена одной копии продукта	250000 руб.
Полная себестоимость одной копии	144789.70 руб.
Планируемый объём продаж	15 копий в год
Объём продаж за 2 года	30 копий
Годовая выручка	3750000 руб.
Выручка за 2 года	7500000 руб.
Прибыль от одной копии	145200 руб.
Годовая прибыль	2178000 руб.
Прибыль за 2 года	4356000 руб.
Срок достижения точки безубыточности по балансу	13–15 месяцев
Чистая приведённая стоимость NPV	1693073.56 руб.
Внутренняя норма доходности IRR	119.62%
Рентабельность инвестиций ROI	201.82%
Экономическая добавленная стоимость EVA	1889350.71 руб.
Срок окупаемости инвестиций	7.95 месяцев

Таким образом, выполненное организационно-экономическое обоснование показывает, что разработка методики и программного инструмента верификации является экономически целесообразной. Несмотря на ограниченный рынок сбыта, специализированный характер продукта обеспечивает достаточный уровень цены, позволяющий покрыть затраты на разработку и внедрение и получить положительный финансовый результат.

ОРГАНИЗАЦИОННО-ПРАВОВАЯ ЧАСТЬ

Организационно-правовой аспект данной работы связан прежде всего с тем, что разработка и применение подобных методик затрагивают сферу информационных технологий, защиты информации и безопасной разработки программного обеспечения. При формировании трасс исполнения, обработке событий, агрегировании журналов и анализе результатов верификации необходимо учитывать требования законодательства Российской Федерации и нормативных документов, регулирующих обращение с информацией, обеспечение её целостности и конфиденциальности, а также требования к процессам создания безопасного программного обеспечения.

Следовательно, организационно-правовая часть работы направлена на анализ нормативных правовых актов и нормативных документов Российской Федерации, которые задают общие принципы обращения с информацией, определяют требования к её защите и формируют правовые и организационные основания для применения методов верификации и безопасной разработки программных средств. Рассмотрение указанных документов позволяет показать, что разрабатываемая методика не противоречит действующему законодательству Российской Федерации и соответствует современным подходам к обеспечению надёжности и безопасности программных систем.

Основные нормативные правовые акты и нормативные документы, рассматриваемые в данной организационно-правовой части, следующие:

- Конституция Российской Федерации;
- Федеральный закон от 27.07.2006 №149-ФЗ «Об информации, информационных технологиях и о защите информации»;
- Указ Президента Российской Федерации от 05.12.2016 №646 «Об утверждении Доктрины информационной безопасности Российской Федерации»;
- ГОСТ Р 56939–2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования»;
- Приказ ФСТЭК России от 01.12.2023 №240 «Об утверждении Порядка проведения сертификации процессов безопасной разработки программного обеспечения средств защиты информации».

Анализ указанных нормативных правовых актов и нормативных документов позволяет определить правовой контекст разработки методики верификации соответствия TLA+ спецификации и исходного кода, а также обосновать её значение как инструмента повышения корректности, надёжности и информационной безопасности программных систем.

12 Конституция Российской Федерации

Конституция Российской Федерации является основой правового регулирования всех общественных отношений на территории государства и обладает высшей юридической силой. В соответствии с частью 1 статьи 15 Конституции Российской Федерации она имеет высшую юридическую силу, прямое действие и применяется на всей территории Российской Федерации. Законы и иные правовые акты, принимаемые в Российской Федерации, не должны противоречить Конституции Российской Федерации [28].

Для рассматриваемой темы Конституция Российской Федерации имеет значение как базовый нормативный акт, закрепляющий фундаментальные принципы обращения с информацией, пределы реализации прав в информационной сфере, а также общие требования, которые должны учитываться при разработке и применении методики верификации соответствия TLA+ спецификации и исходного кода.

Статья 23 Конституции Российской Федерации закрепляет право каждого на неприкосновенность частной жизни, личную и семейную тайну, защиту своей чести и доброго имени, а также право на тайну переписки, телефонных переговоров, почтовых, телеграфных и иных сообщений [28]. Для темы выпускной квалификационной работы данное положение имеет практическое значение, поскольку предлагаемая методика использует инструментирование кода и формирование трасс исполнения. Если в процессе трассировки, журналирования событий или агрегации данных исполнения обрабатываются сведения, позволяющие установить содержание сообщений, действий пользователей или иные данные ограниченного доступа, такие действия должны осуществляться с соблюдением конституционных гарантий неприкосновенности частной жизни и конфиденциальности сообщений.

Статья 24 Конституции Российской Федерации устанавливает, что сбор, хранение, использование и распространение информации о частной жизни лица без его согласия не допускаются [28]. Это положение непосредственно связано с применением методики верификации в реальных информационных системах. Хотя сама методика ориентирована на анализ корректности поведения распределённой реализации и сопоставление её со спецификацией TLA+, на практике формируемые трассы исполнения могут содержать технические сведения, косвенно связанные с субъектами данных, идентификаторами

сеансов, сетевыми сообщениями или иными атрибутами, позволяющими соотнести исполнение системы с конкретными лицами. В связи с этим при проектировании и применении методики необходимо исходить из минимизации обрабатываемых данных и недопустимости включения в трассы избыточной информации о частной жизни.

Статья 29 Конституции Российской Федерации закрепляет право каждого свободно искать, получать, передавать, производить и распространять информацию любым законным способом [28]. Для данной темы это означает, что анализ технической информации о работе программной системы, исследование её поведения, построение формальных моделей и проверка их согласованности с реализацией сами по себе соответствуют конституционному принципу свободы получения и обработки информации, если осуществляются законными способами и не нарушают установленные ограничения. Именно в этих пределах разработка методики верификации выступает как правомерная научно-техническая деятельность, направленная на повышение надёжности и безопасности программного обеспечения.

Вместе с тем часть 3 статьи 17 и часть 3 статьи 55 Конституции Российской Федерации указывают, что осуществление прав и свобод человека и гражданина не должно нарушать права и свободы других лиц, а сами права и свободы могут быть ограничены федеральным законом в той мере, в какой это необходимо для защиты основ конституционного строя, нравственности, здоровья, прав и законных интересов других лиц, обеспечения обороны страны и безопасности государства [28]. Применительно к рассматриваемой работе это означает, что использование средств инструментирования, трассировки и анализа поведения программной системы не может осуществляться произвольно. Если программная система функционирует в среде, где обрабатывается конфиденциальная информация, персональные данные или иные сведения ограниченного доступа, методика верификации должна быть организована таким образом, чтобы не создавать угрозу нарушения прав субъектов данных и не противоречить специальным требованиям федерального законодательства.

Таким образом, Конституция Российской Федерации задаёт базовые правовые ориентиры для разработки и применения методики верификации соответствия TLA+ спецификации и исходного кода. С одной стороны, она допускает осуществление научной, технической и аналитической деятельности

по исследованию поведения программных систем, а с другой стороны — требует соблюдения прав граждан на неприкосновенность частной жизни, конфиденциальность сообщений и законные ограничения при работе с информацией. Следовательно, разрабатываемая методика должна применяться с учётом конституционных принципов законности, соразмерности и защиты прав человека [28].

13 Федеральный закон от 27.07.2006 №149-ФЗ «Об информации, информационных технологиях и о защите информации»

Федеральный закон №149-ФЗ «Об информации, информационных технологиях и о защите информации» является одним из основных нормативных правовых актов, регулирующих отношения в сфере информации, информационных технологий и её защиты. Для темы данной выпускной квалификационной работы этот закон имеет особое значение, поскольку предлагаемая методика верификации соответствия TLA+ спецификации и исходного кода связана со сбором, обработкой, хранением и анализом информации о поведении программной системы [29].

Согласно статье 1 Федерального закона №149-ФЗ, закон регулирует отношения, возникающие при осуществлении права на поиск, получение, передачу, производство и распространение информации, при применении информационных технологий, а также при обеспечении защиты информации [29]. Это позволяет отнести разрабатываемую методику к сфере действия данного закона, поскольку в её рамках используются программные средства трассировки, агрегирования данных исполнения и модельной проверки.

Статья 3 Федерального закона №149-ФЗ закрепляет основные принципы правового регулирования в данной области. К ним относятся законность обращения с информацией, достоверность информации, своевременность её предоставления, а также необходимость защиты информации и соблюдения прав граждан при работе с ней [29]. Применительно к теме работы это означает, что сбор трасс исполнения и их последующий анализ должны осуществляться только в рамках законных целей, а полученные результаты должны быть достоверными и защищёнными от искажения.

Важное значение имеет также статья 5 Федерального закона №149-ФЗ, в соответствии с которой информация рассматривается как объект правовых отношений [29]. Для рассматриваемой методики это означает, что исходный код, формальная спецификация, локальные и глобальные трассы исполнения, а также результаты проверки средствами TLC могут рассматриваться как самостоятельные информационные объекты, в отношении которых должен соблюдаться установленный режим доступа, хранения и использования.

Наиболее значимой для практического применения методики является статья 16 Федерального закона №149-ФЗ, посвящённая защите информации. В ней установлено, что защита информации включает правовые, организационные и технические меры, направленные на предотвращение неправомерного доступа, уничтожения, модифицирования, блокирования, копирования и распространения информации [29]. Для данной работы это особенно важно, поскольку искажение трасс исполнения, их неполнота или несанкционированное изменение могут привести к неверным выводам о соответствии реализации формальной спецификации. Следовательно, при использовании предлагаемой методики необходимо обеспечивать целостность, достоверность и защищённость всех данных, формируемых в процессе верификации.

Кроме того, статья 17 Федерального закона №149-ФЗ закрепляет, что нарушение требований законодательства в области информации, информационных технологий и защиты информации влечёт ответственность в соответствии с законодательством Российской Федерации [29]. Это положение подчёркивает, что при практическом применении методики необходимо учитывать не только технические аспекты трассировки и проверки, но и правомерность обращения с формируемыми данными.

Таким образом, Федеральный закон №149-ФЗ формирует общую правовую основу для разработки и применения методики верификации соответствия TLA+ спецификации и исходного кода. Закон определяет сферу регулирования отношений, связанных с информацией и информационными технологиями, рассматривает информацию как объект правовых отношений и устанавливает требования к её защите. Следовательно, при разработке и применении предлагаемой методики необходимо обеспечивать законность обработки данных исполнения, соблюдение режима доступа к информации и защиту результатов верификации от несанкционированного воздействия [29].

14 Указ Президента Российской Федерации от 05.12.2016 №646 «Об утверждении Доктрины информационной безопасности Российской Федерации»

Указ Президента Российской Федерации от 05.12.2016 №646 утвердил Доктрину информационной безопасности Российской Федерации, которая представляет собой систему официальных взглядов на обеспечение национальной безопасности Российской Федерации в информационной сфере [30]. Для темы данной выпускной квалификационной работы этот документ представляет интерес как нормативная основа, определяющая значение информационной безопасности, надёжности функционирования информационных систем и развития безопасных отечественных информационных технологий.

Доктрина информационной безопасности Российской Федерации рассматривает информационную безопасность как важный элемент национальной безопасности и подчёркивает необходимость устойчивого и бесперебойного функционирования информационной инфраструктуры [30]. Для рассматриваемой темы это имеет особое значение, поскольку методика верификации соответствия TLA+ спецификации и исходного кода направлена на повышение корректности поведения программных систем, выявление расхождений между формальной моделью и реализацией и, как следствие, на снижение риска ошибок в критически важных сценариях работы распределённых систем.

Существенным положением Доктрины является акцент на развитии и совершенствовании информационных технологий, а также на необходимости повышения защищённости и качества программных средств, используемых в информационной инфраструктуре [30]. В этом контексте разрабатываемая методика может рассматриваться как инструмент, способствующий достижению указанных целей, поскольку она позволяет подтверждать допустимость наблюдаемого поведения реализации с точки зрения формальной спецификации и своевременно обнаруживать противоречия между моделью и программным кодом.

Кроме того, в Доктрине отмечается значимость обеспечения защиты прав и свобод человека и гражданина в информационной сфере, а также важность применения организационных и технических мер, направленных на повышение уровня информационной безопасности [30]. Для данной работы это означает, что методика верификации должна рассматриваться не только как средство повышения качества программного обеспечения, но и как часть более общего подхода к обеспечению надёжности и безопасности информационных систем.

Таким образом, Доктрина информационной безопасности Российской Федерации формирует стратегический контекст для разработки и применения методики верификации соответствия TLA+ спецификации и исходного кода. Она подчёркивает значимость обеспечения устойчивого функционирования информационных систем, развития безопасных информационных технологий и использования современных методов, направленных на повышение надёжности и защищённости программного обеспечения [30].

15 ГОСТ Р 56939–2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования»

ГОСТ Р 56939–2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования» является важным нормативным документом для организационно-правового анализа темы данной выпускной квалификационной работы. В отличие от общих нормативных правовых актов, регулирующих сферу информации и её защиты, данный стандарт непосредственно связан с процессом создания безопасного программного обеспечения и устанавливает требования к организации работ на различных этапах его жизненного цикла [31].

Для темы разработки методики верификации соответствия TLA+ спецификации и исходного кода данный стандарт представляет особый интерес, поскольку рассматриваемая методика направлена на подтверждение корректности реализации, выявление расхождений между формальной моделью и программным кодом, а также на повышение надёжности программной системы. Использование формальных спецификаций, трасс исполнения и средств модельной проверки соответствует общей задаче безопасной разработки, заключающейся в предупреждении ошибок, способных привести к нарушениям функционирования, уязвимостям и иным недостаткам программного обеспечения.

ГОСТ Р 56939–2024 устанавливает общие требования к содержанию и порядку выполнения работ, связанных с созданием безопасного программного обеспечения и устранением выявленных недостатков, в том числе уязвимостей [31]. Применительно к рассматриваемой работе это означает, что методика верификации может рассматриваться как один из инструментов, поддерживающих выполнение требований безопасной разработки, поскольку она позволяет анализировать соответствие поведения реализации заданной спецификации и своевременно выявлять противоречия между моделью и исходным кодом.

Практическая значимость данного стандарта для темы выпускной квалификационной работы заключается также в том, что он ориентирует разработчика на системный подход к обеспечению безопасности программного обеспечения. Верификация соответствия реализации формальной

спецификации в этом контексте выступает не как изолированная исследовательская процедура, а как часть более общего процесса обеспечения качества, надёжности и безопасности разрабатываемого программного продукта.

Таким образом, ГОСТ Р 56939–2024 формирует нормативную основу для рассмотрения предлагающейся методики как элемента безопасной разработки программного обеспечения. Его положения подтверждают, что применение средств формального анализа, проверки поведения системы и выявления расхождений между моделью и реализацией соответствует современным подходам к созданию защищённого и надёжного программного обеспечения [31].

16 Приказ ФСТЭК России от 01.12.2023 №240 «Об утверждении Порядка проведения сертификации процессов безопасной разработки программного обеспечения средств защиты информации»

Приказ ФСТЭК России от 01.12.2023 №240 утверждает Порядок проведения сертификации процессов безопасной разработки программного обеспечения средств защиты информации [32]. Для темы данной выпускной квалификационной работы этот документ представляет особый интерес, поскольку он связывает требования безопасной разработки программного обеспечения с процедурами их официального подтверждения.

В контексте рассматриваемой темы значение данного приказа состоит в том, что предлагаемая методика верификации соответствия TLA+ спецификации и исходного кода может рассматриваться как один из практических инструментов, используемых в процессе безопасной разработки программного обеспечения. Методика направлена на выявление расхождений между формальной моделью и реализацией, а значит способствует повышению корректности программного обеспечения и снижению вероятности ошибок, способных повлиять на его безопасность и надёжность.

С практической точки зрения приказ ФСТЭК России №240 показывает, что процессы безопасной разработки подлежат формализации, оценке и подтверждению на соответствие установленным требованиям [32]. Это особенно важно для данной работы, поскольку разработка методики верификации не является изолированной исследовательской задачей, а может быть включена в более широкий процесс обеспечения качества и безопасности программного обеспечения.

Таким образом, приказ ФСТЭК России №240 дополняет положения ГОСТ Р 56939–2024 и позволяет рассматривать предлагаемую методику верификации как часть современного подхода к безопасной разработке программного обеспечения. Применение методов формального анализа, трассировки исполнения и проверки соответствия реализации спецификации согласуется с направлением развития процессов безопасной разработки, которые в ряде случаев подлежат официальной сертификации [32].

17 Заключение

В ходе выполнения организационно-правового анализа были рассмотрены основные нормативные правовые акты и нормативные документы, значимые для разработки методики верификации соответствия TLA+ спецификации и исходного кода. Проведённый анализ показал, что рассматриваемая методика относится к сфере информационных технологий, обработки и защиты информации, а также связана с современными подходами к безопасной разработке программного обеспечения [28—32].

Конституция Российской Федерации формирует базовые правовые принципы, которые должны учитываться при разработке и применении методики: законность обращения с информацией, соблюдение прав граждан на неприкосновенность частной жизни и допустимость ограничений доступа к отдельным категориям сведений только на основании закона [28]. Федеральный закон №149-ФЗ, в свою очередь, определяет общую правовую рамку для работы с информацией, применения информационных технологий и защиты информации, что непосредственно соотносится с процессами трассировки, хранения, агрегации и анализа данных исполнения распределённой системы [29].

Доктрина информационной безопасности Российской Федерации подчёркивает значимость устойчивого функционирования информационной инфраструктуры, развития безопасных информационных технологий и повышения надёжности программных средств [30]. ГОСТ Р 56939–2024 и приказ ФСТЭК России №240 дополняют данную правовую основу с точки зрения безопасной разработки программного обеспечения, закрепляя требования к организации соответствующих процессов и подтверждая значимость методов, направленных на своевременное выявление недостатков и повышение качества программных средств [31; 32].

Таким образом, предлагаемая методика верификации соответствия TLA+ спецификации и исходного кода не противоречит действующему законодательству и нормативным документам Российской Федерации. Напротив, её применение соответствует современным требованиям к надёжности, корректности и безопасности программного обеспечения, а также может рассматриваться как элемент более общего процесса безопасной разработки и контроля качества программных систем [28—32].

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Таненбаум Э., Ван-Стеен М. Распределенные системы. Принципы и парадигмы. — Спб.: Питер, 2003. — С. 877.

2. Качин К., Гуерру Р., Родригес Л. Введение в надежное и безопасное распределенное программирование. — М.: ДМК Пресс, 2011. — С. 512.

3. Петров А. Распределенные данные. Алгоритмы работы современных систем хранения информации. — Спб.: Питер, 2024. — С. 336.

4. Fischer M. J., Lynch N. A., Paterson M. Impossibility of Distributed Consensus with One Faulty Process // J. ACM. — 1985. — Т. 32, № 2. — С. 374—382.

5. Skeen, Dale, and M. Stonebraker. A Formal Model of Crash Recovery in a Distributed System // IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. — 1983. — Т. SE—9, № 3. — С. 219—228.

6. Skeen, Dale, and M. Stonebraker. // IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. — 1983. — Т. SE—9, № 3. — С. 219—228.

7. Lamport, Leslie. The part-time parliament // ACM Transactions on Computer Systems. — 1998. — Т. 16, № 2. — С. 133—169.

8. Lamport, Leslie. Paxos made simple // ACM SIGACT News. — 2001. — Т. 32, № 4. — С. 51—58.

9. Ongaro, Diego, and John Ousterhout. In search of an understandable consensus algorithm // In Proceedings of the 2014 USENIX conference on USENIX Annual Technical Conference. — 2006. — С. 305—320.

10. Lamport L. The Specification Language TLA+. — 2008. — С. 616—620.

11. Henri Habrias, Marc Frappier. Software Specification Methods. — John Wiley and Sons, 2006. — С. 418.

12. Validating Traces of Distributed Programs Against TLA+ Specifications / H. Cirstea [и др.] // Software Engineering and Formal Methods: 22nd International Conference, SEFM 2024. — Aveiro, Portugal : Springer, 11.2024. — С. 126—143. — DOI: 10.1007/978-3-031-77382-2_8.

13. Model-Based Trace-Checking / Y. Howard [и др.] // arXiv preprint arXiv:1111.2825. — 2011. — Нояб. — arXiv: 1111.2825 [cs.SE].

14. Smart Casual Verification of CCF's Distributed Consensus and Consistency Protocols / H. Howard [и др.] // arXiv preprint arXiv:2406.17455. — 2024. — arXiv: 2406.17455 [cs.DC].

15. Havelund K. Using Runtime Analysis to Guide Model Checking of Java Programs // SPIN Model Checking and Software Verification. Т. 1885 / под ред. K. Havelund, J. Penix, W. Visser. — Springer, 2000. — С. 245—264. — (Lecture Notes in Computer Science).

16. Falcone Y., Havelund K., Reger G. A Tutorial on Runtime Verification // Engineering Dependable Software Systems. — 2013. — Т. 34. — С. 141—175.

17. Using Formal Specifications to Monitor and Guide Simulation: Verifying the Cache Coherence Engine of the Alpha 21364 Microprocessor / S. Tasiran [и др.] // Proceedings of the 3rd IEEE Workshop on Microprocessor Test and Verification. — 2002.

18. How Amazon Web Services Uses Formal Methods / C. Newcombe [и др.] // Communications of the ACM. — 2015. — Т. 58, № 4. — С. 66—73.

19. Pressler R. Conjunction Capers: A TLA+ Truffle. — 2020. — URL: https://conf.tlapl.us/2020/. Доклад на TLA+ Conference 2020.

20. Davis A. J. J., Hirschhorn M., Schvimer J. eXtreme Modelling in Practice // Proceedings of the VLDB Endowment. — 2020. — Т. 13, № 9. — С. 1346—1358.

21. Verifying Zookeeper Based on Model-Based Runtime Trace-Checking Using TLA+ / Z. Niu [и др.] // Proceedings of the 7th International Conference on Cyber Security and Information Engineering. — ACM, 2022. — С. 13—18.

22. Model Checking Guided Testing for Distributed Systems / D. Wang [и др.] // Proceedings of the Eighteenth European Conference on Computer Systems. — ACM, 2023. — С. 127—143.

23. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm // 2014 USENIX Annual Technical Conference. — USENIX Association, 2014. — С. 305—319.

24. Compiling Distributed System Models with PGo / F. Hackett [и др.] // Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. — ACM, 2023. — С. 159—175.

25. Foo D., Costea A., Chin W.-N. Protocol Conformance with Choreographic PlusCal // Theoretical Aspects of Software Engineering. Т. 13931. — Springer, 2023. — С. 126—145. — (Lecture Notes in Computer Science).

26. Tretmans J. Test Generation with Inputs, Outputs and Repetitive Quiescence // Software Concepts and Tools. — 1996. — Т. 17, № 3. — С. 103—120.

27. Vaandrager F. W. Model Learning // Communications of the ACM. — 2017. — Т. 60, № 2. — С. 86—95.

28. Конституция Российской Федерации. — 2020. — URL: https://www.consultant.ru/document/cons_doc_LAW_28399/ ; Дата обращения: 01.04.2026. КонсультантПлюс.

29. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации». — 2006. — URL: https://www.consultant.ru/document/cons_doc_LAW_61798/ ; Ред. от 29.12.2025. Дата обращения: 01.04.2026. КонсультантПлюс.

30. Указ Президента Российской Федерации от 05.12.2016 № 646 «Об утверждении Доктрины информационной безопасности Российской Федерации». — 2016. — URL: https://www.consultant.ru/document/cons_doc_LAW_208191/ ; Дата обращения: 01.04.2026. КонсультантПлюс.

31. ГОСТ Р 56939–2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования». — 2024. — URL: https://files.stroyinf.ru/Index/83/83777.htm ; Дата обращения: 01.04.2026. Нормативные базы ГОСТ.

32. Приказ ФСТЭК России от 01.12.2023 № 240 «Об утверждении Порядка проведения сертификации процессов безопасной разработки программного обеспечения средств защиты информации». — 2024. — URL: https://www.consultant.ru/document/cons_doc_LAW_474713/ ; Дата обращения: 01.04.2026. КонсультантПлюс.